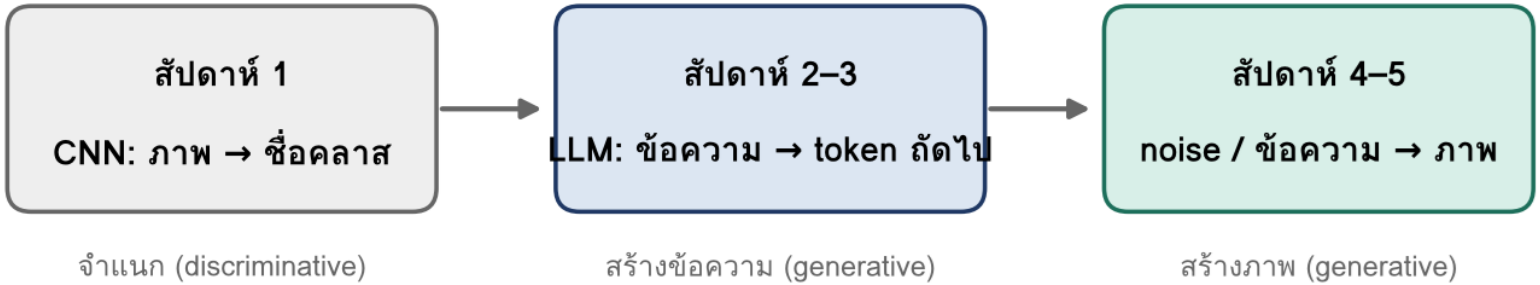
จาก noise สู่ภาพ
หลักการของโมเดลสร้างภาพ

สัปดาห์ 4 · ช่วงเช้า — VAE · GAN · Diffusion

ดร.ธีรพงศ์ อรชร · ANT-DPU

ทบทวน

เส้นทางที่เดินมา และวันนี้จะไปไหน



- สัปดาห์ 1: จำแนก — ภาพเข้า ชื่อคลาสออก (CNN)
- สัปดาห์ 2–3: สร้างข้อความ — LLM ทำนาย token ถัดไป
- วันนี้: สร้างภาพ — ใช้หลักคิดเดียวกับ LLM แต่คำตอบเป็นภาพ

ภาพรวม

วันนี้ทำอะไรบ้าง

1 ช่วงเช้า · 4 ส่วน

1. สร้างภาพ ≠ จำแนกภาพ (และ autoregressive)
2. VAE — บีบแล้วคลาย
3. GAN — เกมสองฝ่าย
4. Diffusion — เดินจาก noise ทีละก้าว

2 ช่วงบ่าย · 4 แล็บ (Colab T4)

แล็บ 1 เทรน VAE
แล็บ 2 เทรน GAN
แล็บ 3 เทรน DDPM + ลอง DDIM
แล็บ 4 Diffusion แบบมีเงื่อนไข

ชุดข้อมูลเดียวกันทั้งวัน: Sprites 16×16

3 ปลายทาง

เข้าใจว่า Stable Diffusion (สัปดาห์หน้า) ประกอบจาก 3 ชิ้นที่เราเทรนเองวันนี้

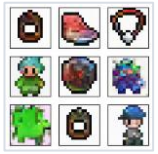
การบ้าน: "3 โมเดล 1 ชุดข้อมูล"



VAE



GAN



Diffusion บน sprites



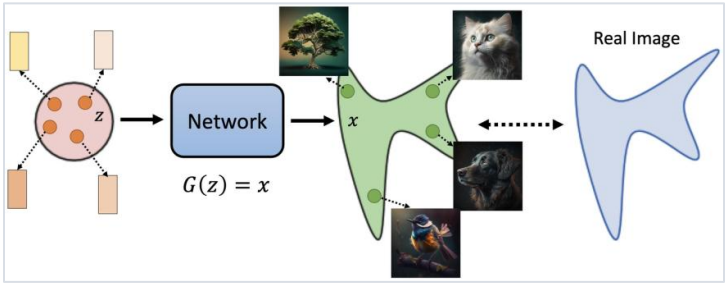
Stable Diffusion (สัปดาห์หน้า)

ภาพ: Siyu Zhu (Fudan) Lecture 2 หน้า 81 · Lecture 3 หน้า 33 · Lecture 6 หน้า 44 · Lecture 4&5 หน้า 47

เส้นด้าย

เส้นด้ายของวัน

โมเดลสร้างภาพ = ฟังก์ชันที่เรียน "หน้าตาของภาพที่เป็นไปได้" แล้วสุ่มภาพใหม่ออกมา — สามโมเดลวันนี้ต่างกันแค่ "วิธีสุ่ม"



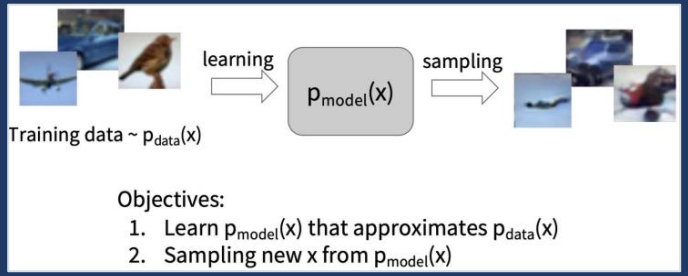
VAE: กระโดดจากรหัส z ทีเดียว · GAN: กระโดดทีเดียว แต่มีกรรมการจับผิด · Diffusion: เดินจาก noise ทีละก้าว
(เทียบกับสัปดาห์ 3: LLM = ฟังก์ชันทำนาย token ถัดไป — วิธีสุ่มคือ 'ทีละ token')

ภาพ: Siyu Zhu (Fudan) Lecture 4&5 หน้า 3 · ต้นฉบับ Hung-yi Lee, NTU 2023

ส่วนที่ 1

สร้างภาพ ≠ จำแนกภาพ

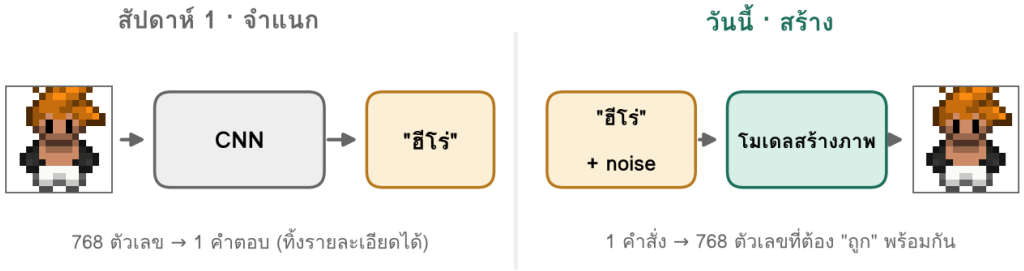
ทำไมการ 'สร้าง' ถึงเป็นโจทย์คนละแบบกับสัปดาห์ 1 · และโมเดลสร้างภาพมีกี่ตระกูล



ภาพ: Siyu Zhu (Fudan) Lecture 2 หน้า 20 · ดัดแปลงตาม Stanford University 2024

1 · สร้าง ≠ จำแนก

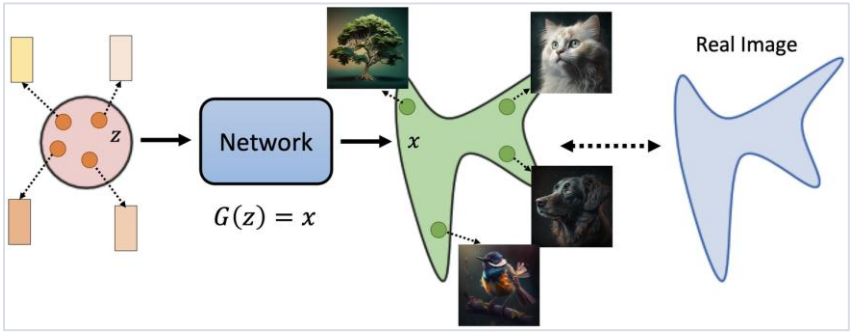
กลับทิศจากสัปดาห์ 1

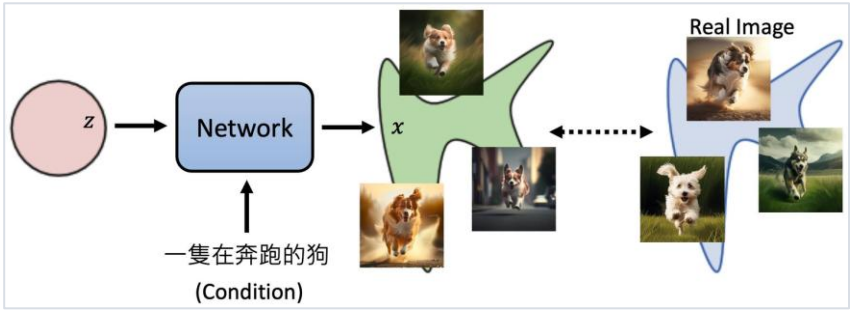


ทิศทางกลับกัน: จำแนก = ย่อข้อมูล · สร้าง = ขยายข้อมูล

- สัปดาห์ 1: ภาพ 768 ตัวเลข → 1 คำตอบ ทิ้งรายละเอียดได้ (CNN ไม่สนว่าฮีโร่ใส่เสื้อสีอะไร)
- วันนี้: 1 คำสั่ง → 768 ตัวเลขที่ต้องถูกพร้อมกัน ทิ้งอะไรไม่ได้เลย
- สรุป: จำแนก = ย่อข้อมูล

ภาพในหัว: เครือข่ายแปลง 'การกระจายง่าย ๆ' ให้เป็น 'การกระจายของภาพ'

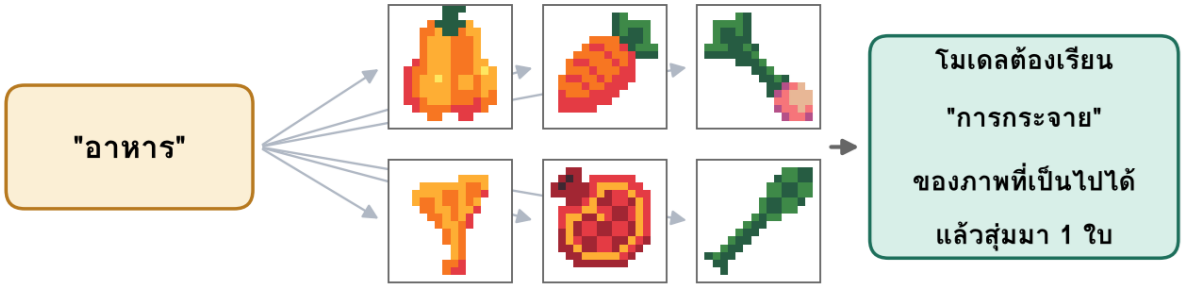




- ซ้าย: สุ่ม z จากการกระจายง่าย ๆ (วงกลม) → Network → ได้ภาพ x
- ภาพที่สร้างได้ทั้งหมด = รูปทรงสีเขียว ภาพจริง = สีฟ้า — เป้าหมายคือทำให้ทับกัน
- ขวา: ใส่เงื่อนไข ('สุนัขกำลังวิ่ง') → ภาพต้องอยู่ในส่วนที่ตรงเงื่อนไข = text-to-image สัปดาห์หน้า

ภาพ: Siyu Zhu (Fudan) Lecture 4&5 หน้า 3, 4 · ดัดแปลงโดย Hung-yi Lee, NTU 2023

ปัญหาใหญ่: คำตอบที่ถูกมีได้หลายแบบ

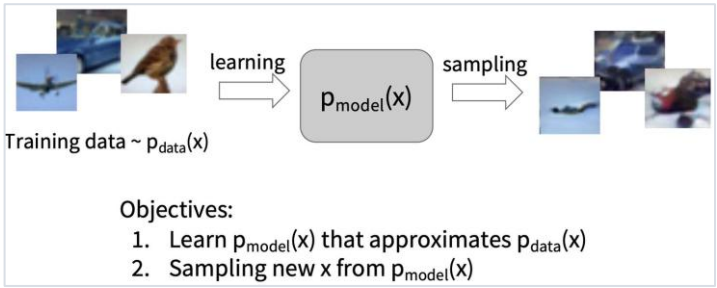


ถูกทั้งหมด — ไม่มี "เฉลย" เดียว

- "อาหาร" คำเดียว แต่ภาพที่ถูกมีได้นับไม่ถ้วน — ไม่มีเฉลยเดียวให้เทียบ MSE ตรง ๆ
- สิ่งที่ต้องเรียนคือ 'การกระจาย' (distribution) ว่าภาพหน้าตาแบบไหนเป็นไปได้มาก/น้อย
- แล้ว 'สุ่ม' ออกมา 1 ใบ — เหมือน LLM ที่สุ่ม token จากการกระจาย ไม่ได้เลือกคำเดียวตายตัว

"เรียนการกระจาย" แปลว่าอะไร (อุปมา)

- **ครูสอนวาดการ์ตูน ไม่ได้ให้ท่องจำภาพทีละใบ แต่ให้จับ "สไตล์": สัดส่วน สี ท่าทางที่พบบ่อย**
  - ลูกศิษย์ที่เรียนรู้สไตล์แล้ว วาดตัวละครใหม่ที่ไม่เคยเห็นได้ — และวาดได้หลายแบบไม่ซ้ำ
- โมเดลสร้างภาพก็เหมือนกัน: ไม่ได้เก็บภาพในชุดข้อมูล แต่เก็บ "กฎว่าภาพแบบไหนสมเหตุสมผล"
  - ตอนใช้งาน: โยนความสุ่ม (noise) เข้าไป → โมเดล 'ปั้น' ความสุ่มนั้นให้กลายเป็นภาพที่เข้ากับกฎ
- โมเดลที่ดี = (1) ภาพคมสมจริง (2) หลากหลาย ไม่ซ้ำ (3) ตรงเงื่อนไขที่สั่ง — สามอย่างนี้จะเป็นเกณฑ์เทียบทั้งวัน

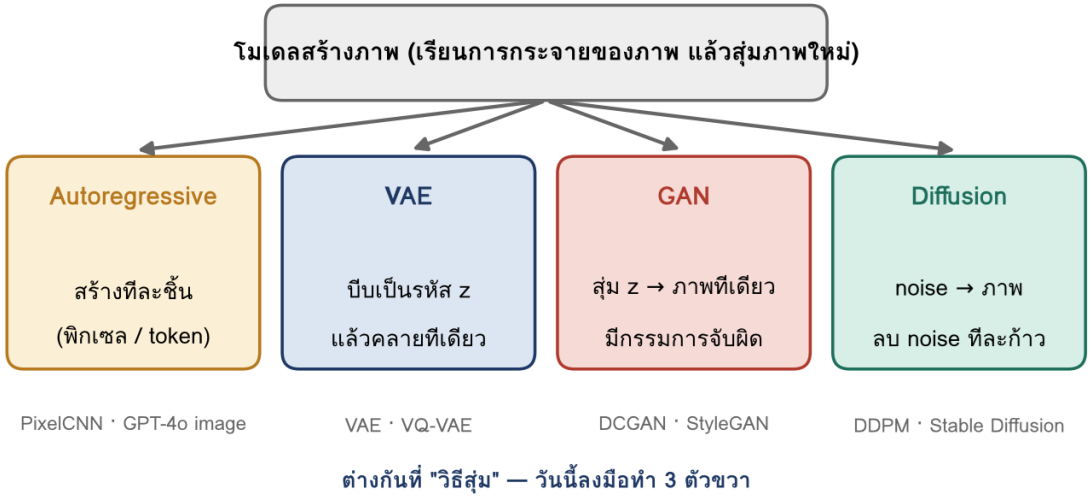


เรียน p_model จากภาพฝึก แล้วสุ่มภาพใหม่

● เกณฑ์ 3 ข้อ: คุณภาพ · ความหลากหลาย · ตรงเงื่อนไข — สัปดาห์หน้าจะใส่ตัวเลข (FID, CLIP score) ให้

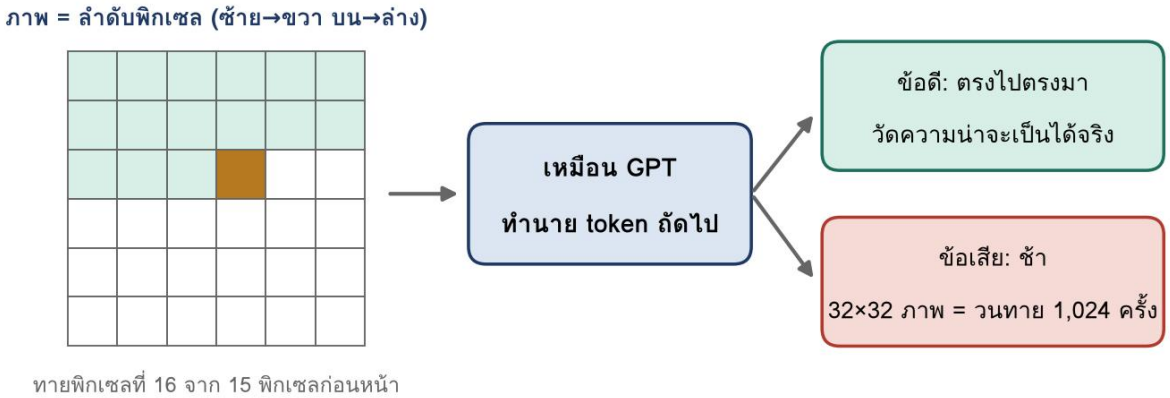
ภาพ: Siyu Zhu (Fudan) Lecture 2 หน้า 20 · ต้นฉบับ Stanford University 2024

แผนที่ตระกูลโมเดลสร้างภาพ



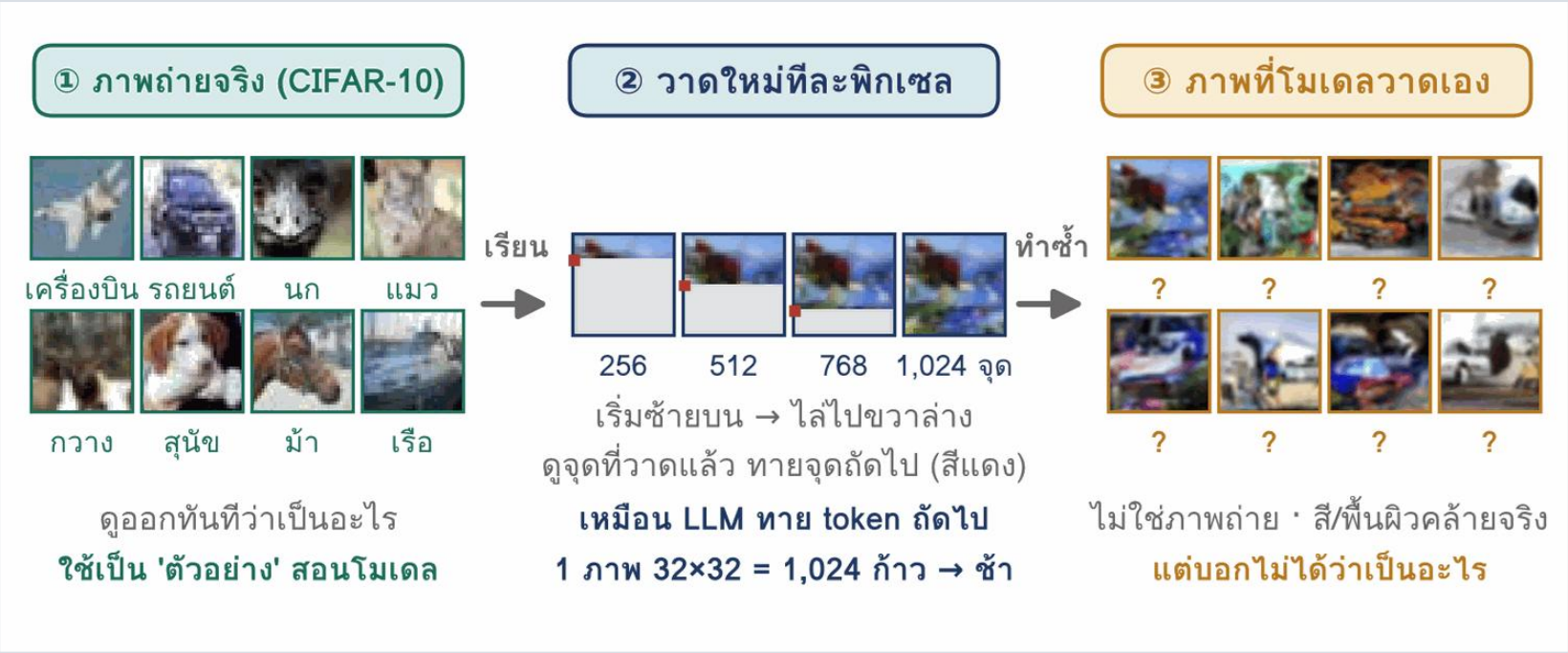
- ทุกตระกูลเรียน 'การกระจาย' เหมือนกัน — ต่างกันที่วิธีสุ่มภาพออกมา
- Autoregressive: สร้างทีละชิ้น (ญาติของ LLM)
- VAE / GAN: สุ่มรหัส z แล้วแปลงเป็นภาพทีเดียว

Autoregressive บนพิกเซล = GPT ของพิกเซล



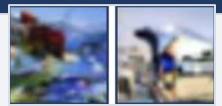
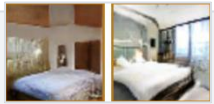
- PixelRNN / PixelCNN (2016): เรียงพิกเซลเป็นลำดับ แล้วทำนายพิกเซลถัดไปจากพิกเซลก่อนหน้า — สูตรเดียวกับ LLM สัปดาห์ 2
- ข้อดี: ตรงไปตรงมา คำนวณความน่าจะเป็นของภาพได้จริง ๆ
- ข้อเสีย: สร้างช้ามาก เพราะต้องวนทีละพิกเซล (ภาพ 32×32 = 1,024 รอบ)

PixelRNN: เรียนจากภาพจริง แล้ววาดใหม่ทีละพิกเซล



ภาพจริง: ชุดข้อมูล CIFAR-10 (Krizhevsky 2009) · ภาพที่โมเดลวาด: PixelRNN, van den Oord et al. 2016 ผ่าน Siyu Zhu (Fudan) Lecture 2 หน้า 35 (ดัดแปลงตามฉบับ Stanford University)

ทำไมต้องมีมากกว่าหนึ่งตระกูล — ไม่มีใครได้ครบทุกข้อ

ภาพที่โมเดลสร้างเอง*		คุณภาพภาพ	ความหลากหลาย	เทรนง่าย	สร้างเร็ว
	**Autoregressive**	รุ่นแรกหยาบ รุ่นใหม่ดี	ดี	ง่าย (เหมือน LLM)	ช้ามาก
	**VAE**	เบลอ	ดี	ง่าย	เร็วมาก
	**GAN**	คมมาก	เสี่ยงซ้ำ (mode collapse)	ยาก แกว่ง	เร็วมาก
	**Diffusion**	คมมาก	ดี	ง่าย	ช้า (แก้ได้)

*ภาพคอลัมน์ซ้ายทุกภาพเป็นภาพที่โมเดลสร้าง ไม่ใช่ภาพถ่าย (ต่างปี ต่างชุดข้อมูล: PixelRNN 2016 · VAE หน้าคน · DCGAN 2015 · SD + ControlNet 2023)

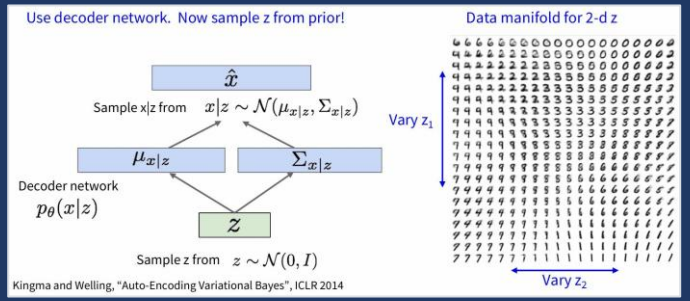
● วันนี้จะเห็นทุกช่องในตารางนี้ด้วยตาตัวเองจากแล็บ — จำไว้แล้วกลับมาเช็กตอนบ่าย

ภาพ: Siyu Zhu (Fudan) Lecture 2 หน้า 35, 81 · Lecture 3 หน้า 33 (ดัดแปลงจาก Stanford University 2024) · Lecture 4&5 หน้า 52 (ControlNet, Zhang et al. 2023)

ส่วนที่ 2

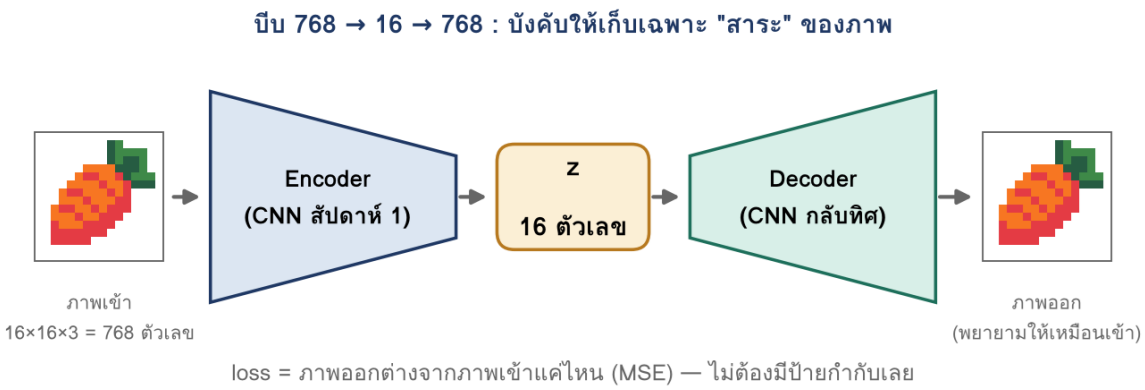
VAE — บีบภาพเป็นรหัสสั้น ๆ แล้วคลายกลับ

Autoencoder · ทำไมต้อง 'variational' · ทำไมภาพเบลอ · VQ-VAE และ 'vocab ของภาพ'



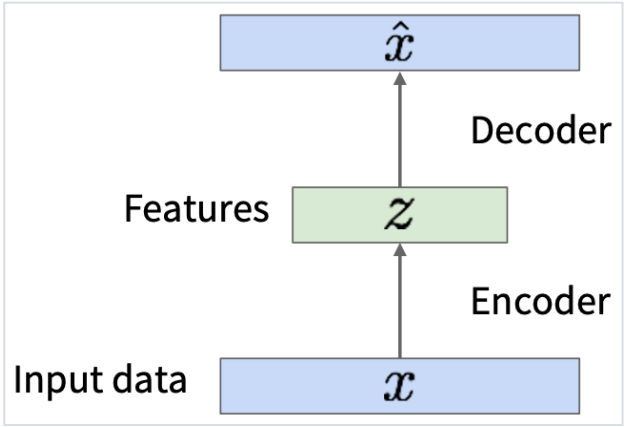
ภาพ: Siyu Zhu (Fudan) Lecture 2 หน้า 78 · ต้นฉบับ Stanford University 2024

Autoencoder: บีบแล้วคลาย



- Encoder = CNN สัปดาห์ 1 ที่ตัดชั้น classifier ออก แล้วให้ออก 'รหัส z' สั้น ๆ (16 ตัวเลข)
- Decoder = CNN กลับทิศ: ขยายรหัส 16 ตัวกลับเป็นภาพ 768 ตัวเลข
- loss = ภาพออกต่างจากภาพเข้าแค่ไหน (MSE) — ไม่ต้องมีป้ายกำกับเลย

Autoencoder ของจริง: ภาพจริง vs ภาพที่คลายกลับ

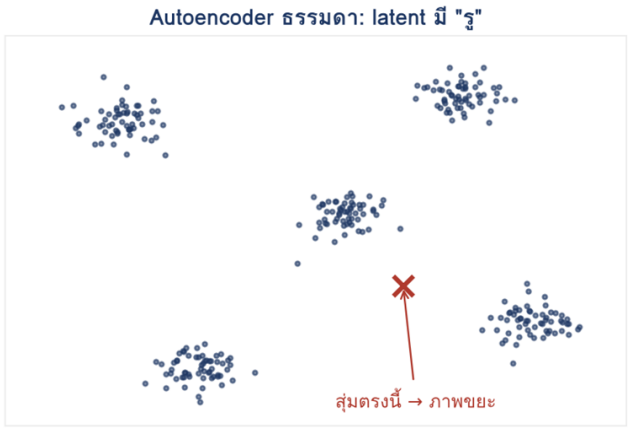




- ซ้าย: x → Encoder → z (features) → Decoder → x̂
- ขวา: ล่าง = ภาพจริง CIFAR-10
- บน = ภาพที่คลายกลับจาก z (encoder/decoder อย่างละ 4 ชั้น)
- สี ท่าทาง ตำแหน่งถูก แต่รายละเอียดหาย — นี่คือ 'สาระ' ที่ z เก็บไว้

ภาพ: Siyu Zhu (Fudan) Lecture 2 หน้า 42 · ต้นฉบับ Stanford University 2024

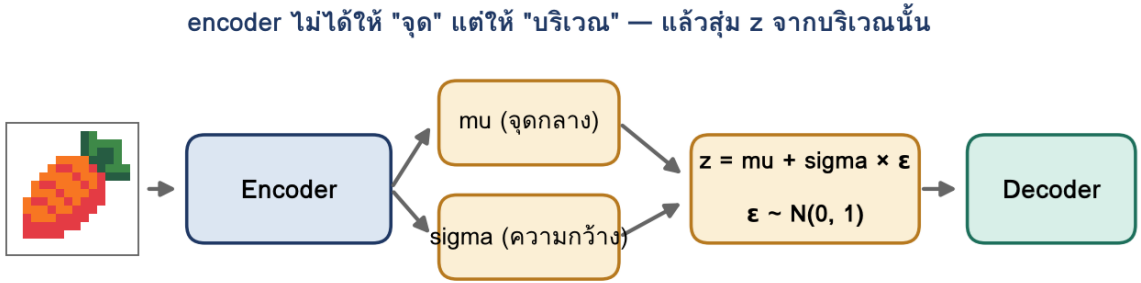
แล้วเอา Autoencoder มาสร้างภาพใหม่ได้ไหม?





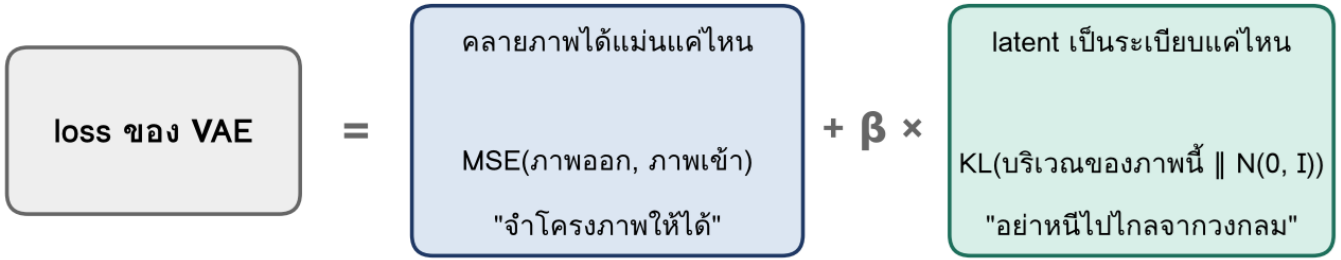
- ไอเดีย: สุ่ม z มั่ว ๆ แล้ว decode → น่าจะได้ภาพใหม่
- ปัญหา: autoencoder ธรรมดาวางรหัสภาพเป็นกระจุก ๆ ห่างกัน ระหว่างกระจุกคือ 'รู' — สุ่มลงรูได้ภาพขยะ
- VAE แก้โดยบังคับให้รหัสทั้งหมดรวมกันเป็น 'วงกลม' เต็ม ๆ ไม่มีรู → สุ่มตรงไหนก็ได้ภาพ

VAE: encoder ให้ 'บริเวณ' ไม่ใช่ 'จุด'



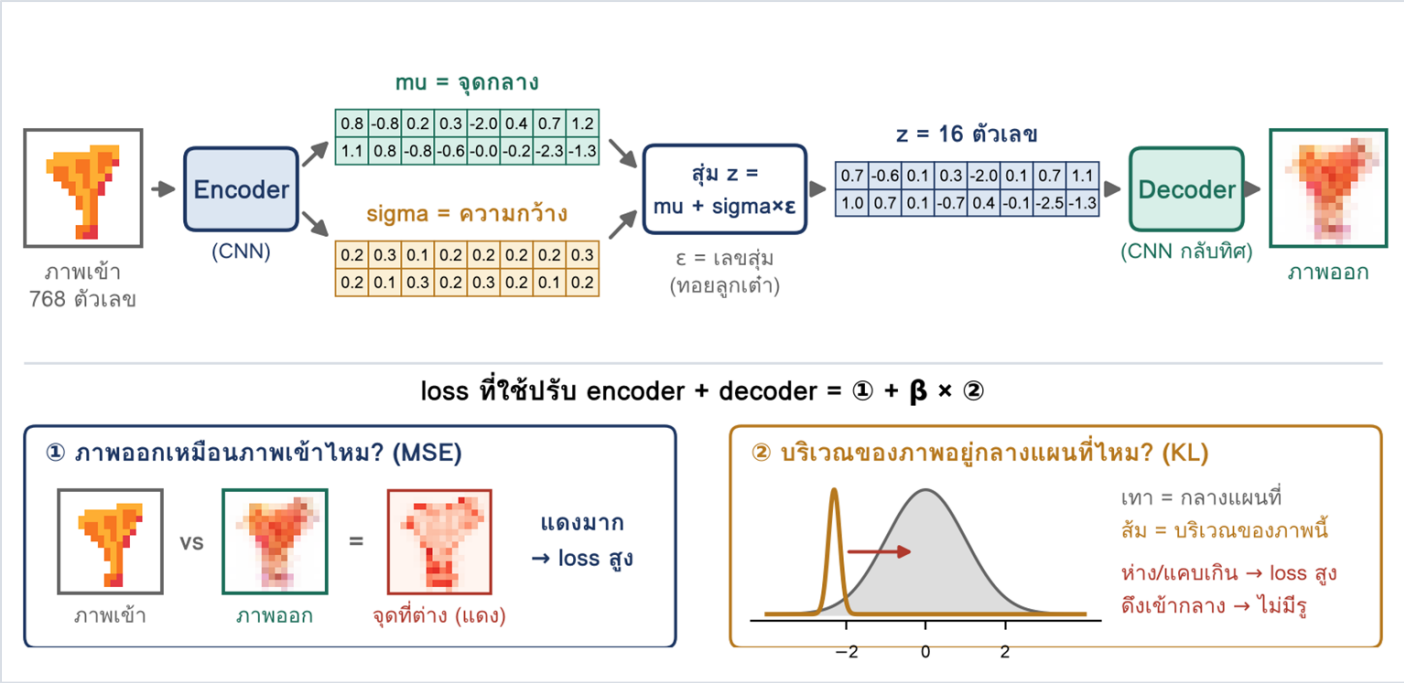
- encoder ออก 2 อย่าง: mu (จุดกลาง) และ sigma (ความกว้างของบริเวณ)
- ตอนเทรน สุ่ม z จากบริเวณนั้น: z = mu + sigma × ε โดย ε สุ่มจาก N(0,1)
- ทริกสำคัญ: ความสุ่มอยู่ที่ ε นอกโมเดล → backprop ผ่าน mu, sigma ได้ (reparameterization trick)

loss ของ VAE มี 2 ส่วน



- ส่วนแรก: คลายภาพได้แม่นแค่ไหน (MSE) — เหมือน autoencoder
- ส่วนสอง: KL divergence = บริเวณของภาพนี้หนีไปไกลจากวงกลมมาตรฐาน N(0, I) แค่ไหน — ค่าปรับที่กันไม่ให้เกิด 'รู'
- β คือตัวถ่วงน้ำหนักระหว่างสองอย่าง (β สูง = latent เป็นระเบียบแต่ภาพเบลอกว่า)

VAE ทั้งระบบ: encoder → สุ่ม z → decoder → loss 2 ส่วน



ภาพ: VAE ที่เทรนจริงบน sprite 10,000 ภาพ (โครงเดียวกับแล็บ 1, เทรนบน CPU) — แทนภาพสมการ Lecture 2 หน้า 75

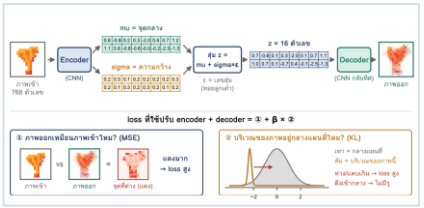
โค้ดที่จะเทรนตอนบ่าย (แล็บ 1)

```
class VAE(nn.Module):
    def __init__(self, z_dim=Z_DIM):
        super().__init__()
        # encoder: 16×16 → 8×8 → 4×4 → 64×4×4 = 1024 ตัวเลข
        self.enc = nn.Sequential(
            nn.Conv2d(3, 32, 4, 2, 1), nn.ReLU(),
            nn.Conv2d(32, 64, 4, 2, 1), nn.ReLU(), nn.Flatten())
        self.fc_mu = nn.Linear(1024, z_dim)         # จุดกลางของบริเวณ
        self.fc_logvar = nn.Linear(1024, z_dim)     # ความกว้าง (log sigma²)
        # decoder: z → 4×4 → 8×8 → 16×16
        self.dec_fc = nn.Linear(z_dim, 1024)
        self.dec = nn.Sequential(
            nn.ConvTranspose2d(64, 32, 4, 2, 1), nn.ReLU(),
            nn.ConvTranspose2d(32, 3, 4, 2, 1), nn.Tanh())

    def encode(self, x):
        h = self.enc(x)
        return self.fc_mu(h), self.fc_logvar(h)

    def reparameterize(self, mu, logvar):
        # z = mu + sigma × ε   (สุ่มแบบที่ยัง backprop ผ่าน mu, sigma ได้)
        return mu + torch.exp(0.5 * logvar) * torch.randn_like(mu)
```

- **enc = Conv 2 ชั้น 16→8→4 (เหมือนสัปดาห์ 1)**
- fc_mu / fc_logvar = จุดกลางและความกว้าง
- **reparameterize = z = mu + sigma × ε**
- dec = ConvTranspose กลับทิศ 4→8→16 + Tanh (ต่อใน notebook)
  - logvar แทน sigma เพื่อให้ค่าเป็นบวกเสมอ: sigma = exp(0.5·logvar)
  - Z_DIM = 16 ตัวเลข



ภาพ: VAE ที่เทรนจริงบน sprite (หน้าก่อน)

ผลที่จะได้เห็น: VAE สร้างภาพได้ แต่ 'เบลอ'

① ทำไมเบลอ: เมื่อคำตอบที่ถูกมีได้ 2 แบบ



ไม่แน่ใจว่าจะหันทางไหน → ตอบกลาง ๆ ให้ผิดน้อยที่สุด

= ภาพซ้อนกันสองแบบ ขอบไม่คม (เบลอ)

② ของจริง: VAE ที่เทรนบน sprite

แถวบน = ภาพจริง · แถวล่าง = ภาพที่ VAE คลายกลับ



รูปร่างและสีถูก แต่ขอบฟุ้ง เส้นเล็ก ๆ หาย

GAN (ส่วน 3) และ Diffusion (ส่วน 4) เลี่ยงปัญหานี้ได้ เพราะไม่ได้ให้คะแนนด้วยการเทียบทีละพิกเซลแบบ MSE ตรง ๆ

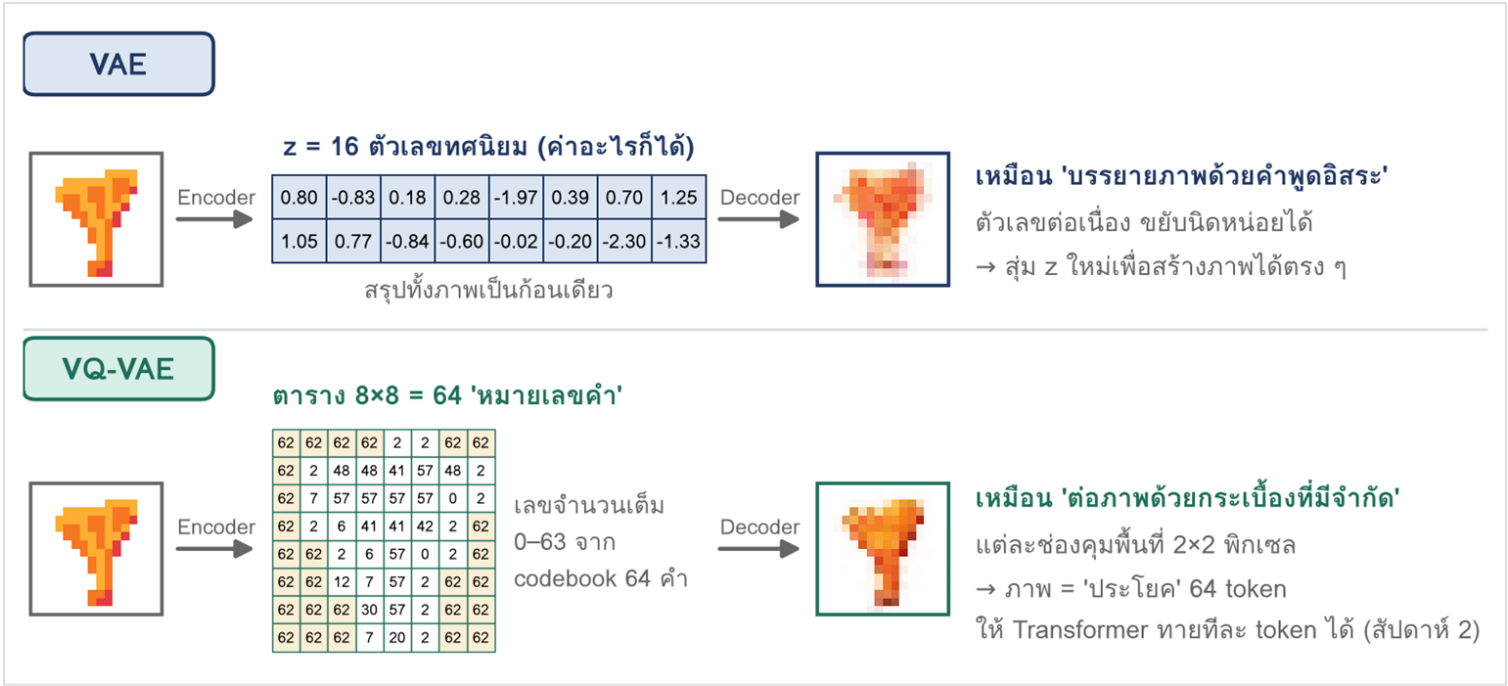
ภาพ: VAE / VQ-VAE ที่เทรนจริงบน sprite 10,000 ภาพ (โครงเดียวกับแล็บ 1, เทรนบน CPU)

ภาพที่ VAE สร้างได้จริง: latent ต่อเนื่อง + ภาพใหม่ที่เบลอ



ภาพ: VAE / VQ-VAE ที่เทรนจริงบน sprite 10,000 ภาพ (โครงเดียวกับแล็บ 1, เทรนบน CPU)

VQ-VAE ต่างจาก VAE ตรงไหน: ตัวเลขอิสระ vs หมายเลขคำ



ภาพ: VAE / VQ-VAE ที่เทรนจริงบน sprite 10,000 ภาพ (โครงเดียวกับแล็บ 1, เทรนบน CPU)

VQ-VAE ทีละขั้น: ภาพ → หมายเลขคำ → ภาพ



① ภาพเข้า
16×16 พิกเซล

Encoder



② ตาราง 8×8 ช่อง
ช่องละ 16 ตัวเลขทศนิยม

③ แต่ละช่อง
หาคำที่ใกล้สุด

62	62	62	62	2	2	62	62
62	2	48	48	41	57	48	2
62	7	57	57	57	57	0	2
62	2	6	41	41	42	2	62
62	62	2	6	57	0	2	62
62	62	12	7	57	2	62	62
62	62	62	30	57	2	62	62
62	62	62	7	20	2	62	62

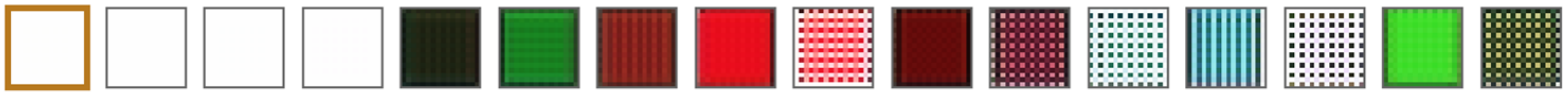
④ หมายเลขคำ 64 ตัว
= 'ประโยค' ของภาพนี้

⑤ เปิด codebook
เลข → เวกเตอร์
แล้วเข้า Decoder



⑥ ภาพออก
คมกว่า VAE

codebook = พจนานุกรม 64 'คำภาพ' ที่โมเดลเรียนเอง — ภาพเล็กคือหน้าตาของคำนั้น ถ้าระบายทั้งภาพด้วยคำเดียว



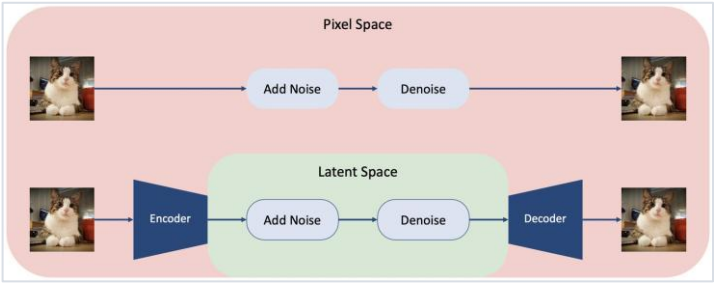
#62 #2 #12 #8 #61 #10 #54 #50 #51 #1 #14 #32 #56 #58 #36 #28

แสดง 16 คำที่ใช้บ่อยสุดจาก 64 คำ · กรอบส้ม = คำ #62 'พื้นหลังขาว' (ช่องสีส้มในตารางข้อ ④)

ภาพ: VAE / VQ-VAE ที่เทรนจริงบน sprite 10,000 ภาพ (โครงเดียวกับแล็บ 1, เทรนบน CPU) — แนวคิดจาก van den Oord et al. 2017 (Siyu Zhu Lecture 2 หน้า 83)

VAE ไปอยู่ที่ไหนในของจริง

- **Stable Diffusion ใช้ VAE บีบภาพ 512×512×3 ให้เหลือ latent 64×64×4 ก่อน (เล็กลง 48 เท่า) แล้วค่อยทำ diffusion บน latent**
  - เหตุผล: diffusion บนพิกเซลเต็ม ๆ ช้าและกินหน่วยความจำมาก — บีบก่อนแล้วค่อยทำจะเร็วกว่าหลายสิบเท่า
- VAE ตัวนั้น latent ต่อเนื่อง (KL) แต่เทรนด้วยสูตร loss ของ VQ-GAN: มี 'กรรมการ' แบบ GAN + perceptual loss (วัดความต่างด้วยฟีเจอร์ CNN แทนพิกเซล) เพื่อให้ภาพคม — ส่วน 3 จะเล่าว่ากรรมการคืออะไร
- VQ-VAE: image token สำหรับโมเดล autoregressive (DALL·E 1; โมเดลภาพของ GPT-4o ก็เป็น autoregressive แต่ไม่เปิดเผยตัว tokenizer)
- งานวิจัย: anomaly detection (ภาพที่ VAE คลายกลับไม่ได้ = ผิดปกติ), การบีบอัด, การสร้างข้อมูลสังเคราะห์



บน: diffusion บนพิกเซล · ล่าง: บีบด้วย VAE ก่อน (latent)

ตอนบ่ายแล็บ 1: เทรน VAE บน sprite แล้วดู latent 'แผนที่ของภาพ' — สิ่งเดียวกับที่ Stable Diffusion ทำงานอยู่ข้างใน

ภาพ: Siyu Zhu (Fudan) Lecture 4&5 หน้า 26 · ต้นฉบับ Mike Shou, NUS

ส่วนที่ 3

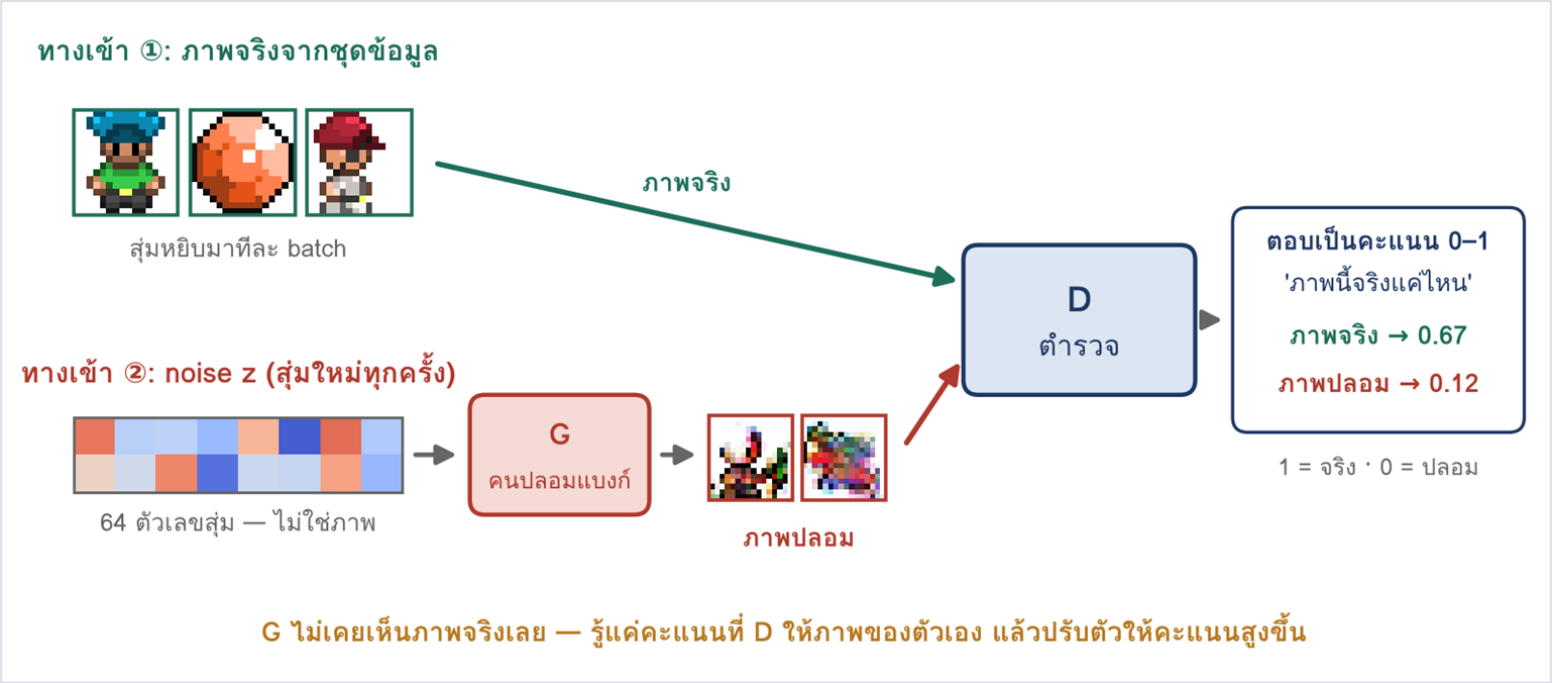
GAN — เกมสองฝ่าย: คนปลอมแบงก์ vs ตำรวจ

ไม่ต้องมีเฉลยพิกเซล · เทรนสลับกันสองขั้น · ภาพคม แต่เทรนยากและเสี่ยง mode collapse



ภาพ: Siyu Zhu (Fudan) Lecture 3 หน้า 33 · ต้นฉบับ Stanford University 2024

เกมสองฝ่าย: ภาพจริงเข้าทางไหน noise เข้าทางไหน



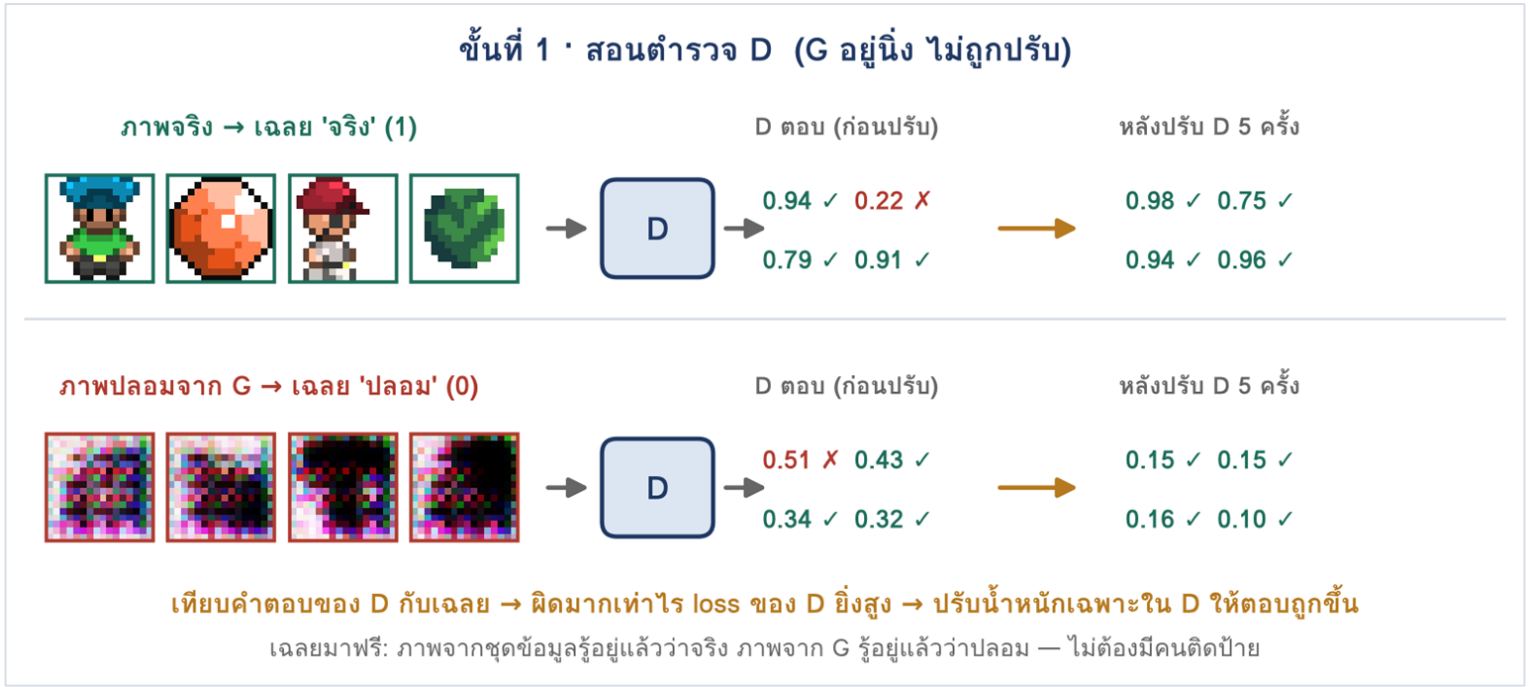
ภาพ: DCGAN ที่เทรนจริงบน sprite 10,000 ภาพ (โครงเดียวกับแล็บ 2, เทรนบน CPU)

จุดพลิก: VAE มีเฉลยพิกเซล — GAN มีแค่คะแนนจาก D



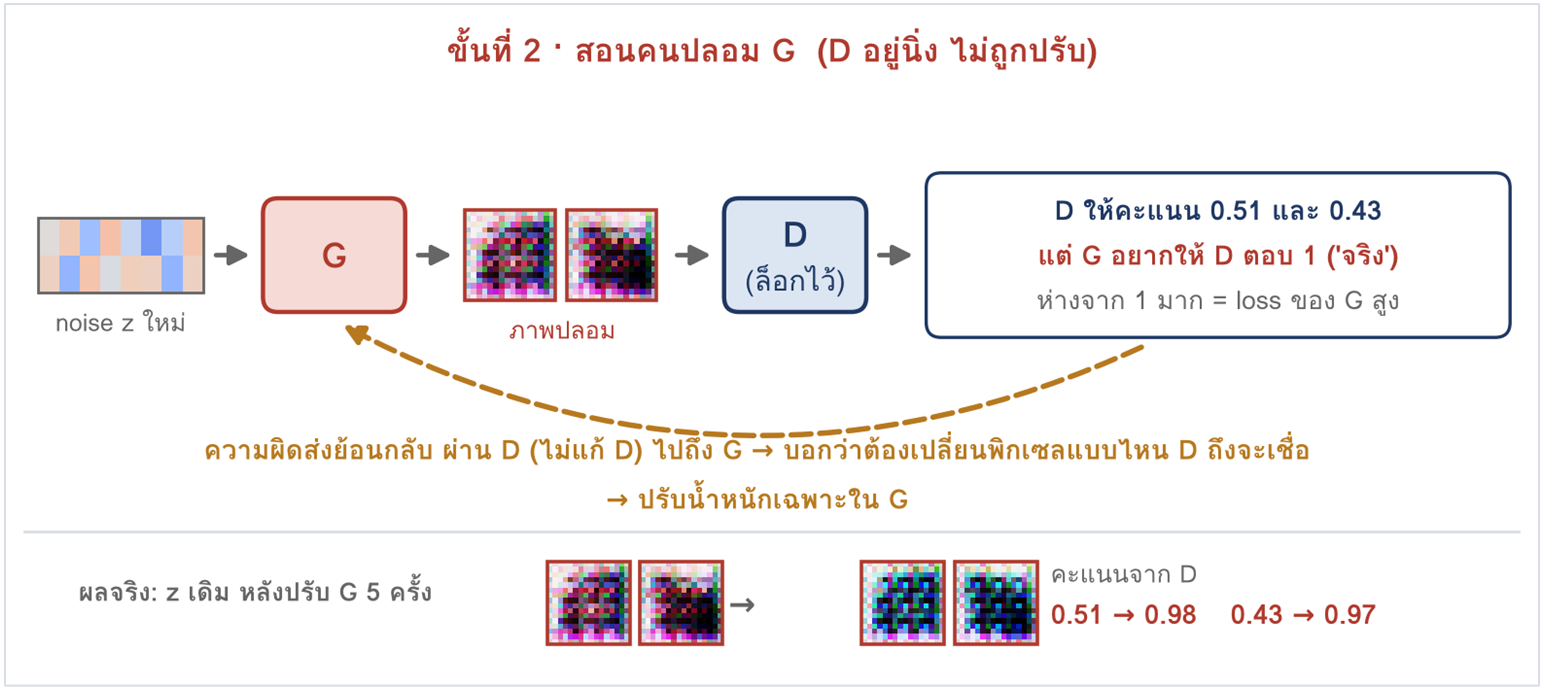
ภาพ: DCGAN ที่เทรนจริงบน sprite 10,000 ภาพ (โครงเดียวกับแล็บ 2, เทรนบน CPU)

ขั้นที่ 1: สอนตำรวจ D ให้แยกภาพจริงกับภาพปลอม



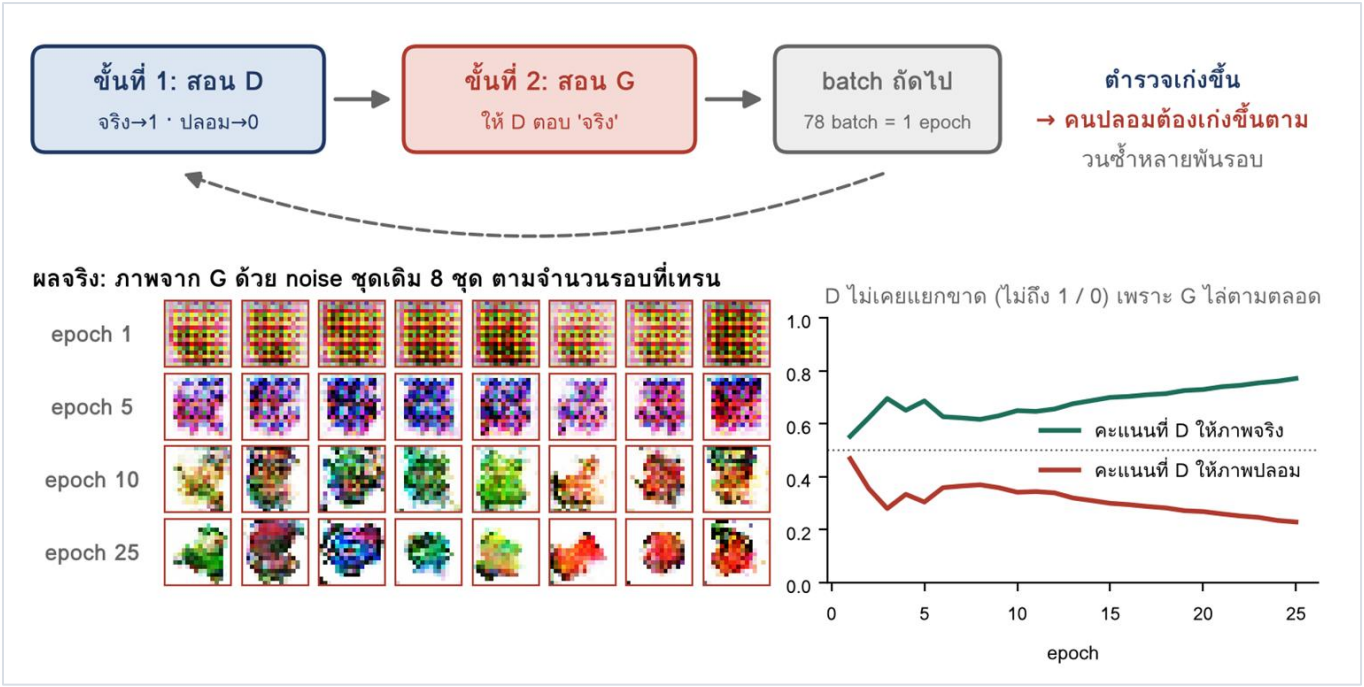
ภาพ: DCGAN ที่เทรนจริงบน sprite 10,000 ภาพ (โครงเดียวกับแล็บ 2, เทรนบน CPU)

ขั้นที่ 2: สอนคนปลอม G ผ่านคะแนนของ D



ภาพ: DCGAN ที่เทรนจริงบน sprite 10,000 ภาพ (โครงเดียวกับแล็บ 2, เทรนบน CPU)

วนสองขั้นนี้ทุก batch: ของปลอมเก่งขึ้นเรื่อย ๆ

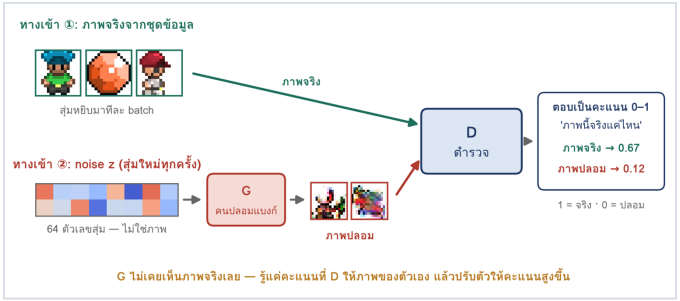


ภาพ: DCGAN ที่เทรนจริงบน sprite 10,000 ภาพ (โครงเดียวกับแล็บ 2, เทรนบน CPU) — อัลกอริทึมจาก Goodfellow et al. 2014

โค้ดที่จะเทรนตอนบ่าย (แล็บ 2)

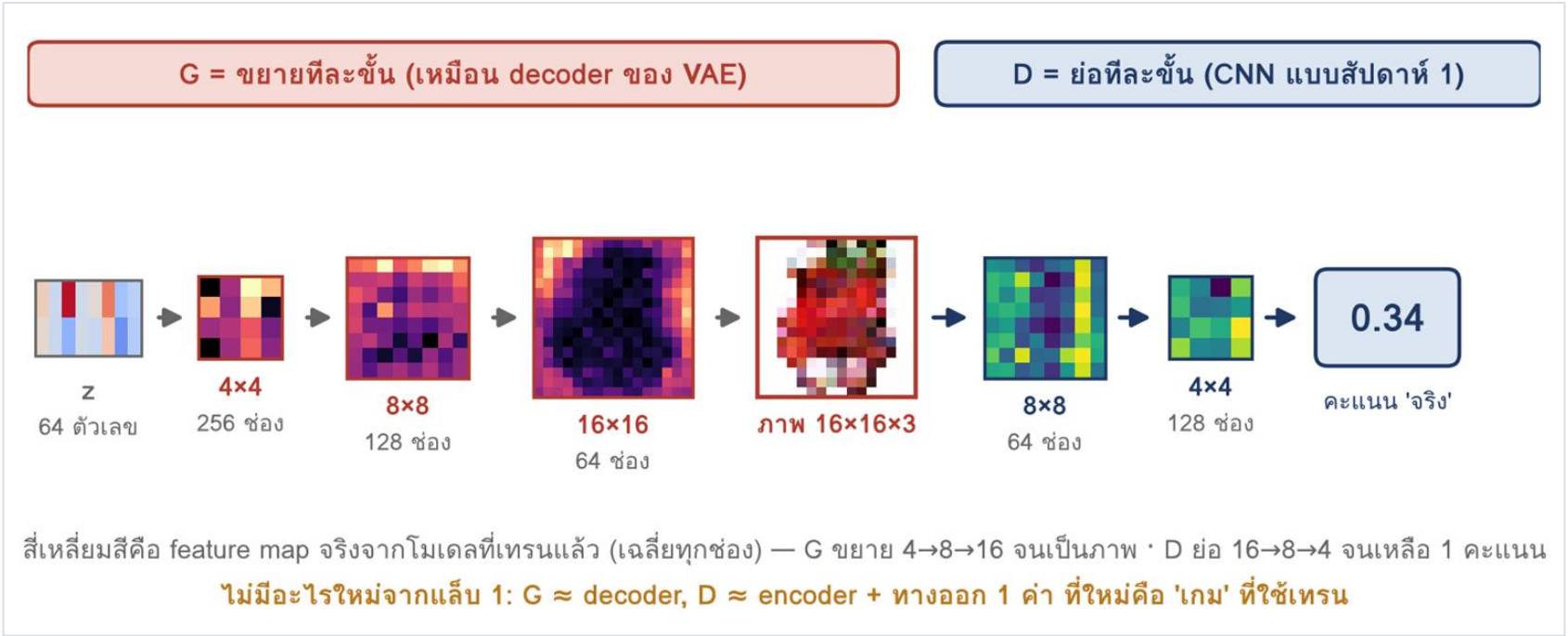
```
# --- (1) ฝึกกรรมการ D ---
fake = G(torch.randn(BATCH, NZ, device=device))
lossD = bce(D(xb), real_lbl) + bce(D(fake.detach()), fake_lbl)
optD.zero_grad(); lossD.backward(); optD.step()
# --- (2) ฝึกคนปลอม G: อยากให้ D ตอบ "จริง" ---
lossG = bce(D(fake), real_lbl)
optG.zero_grad(); lossG.backward(); optG.step()
```

- **(1) D: BCE(D(จริง), 1) + BCE(D(ปลอม), 0)**
  - fake.detach() = ไม่ให้ gradient ของขั้น 1 ไหลเข้า G
- **(2) G: BCE(D(G(z)), 1) — 'อยากให้ตำรวจเชื่อ'**
  - optD / optG แยกกัน: Adam lr 2e-4, betas (0.5, 0.999) — ค่ามาตรฐานของ DCGAN
- ทั้งสองขั้นทำทุก batch สลับกัน



ภาพ: DCGAN ที่เทรนจริงบน sprite (สไลด์ 28)

DCGAN: ข้างใน G และ D คือ CNN



ภาพ: DCGAN ที่เทรนจริงบน sprite 10,000 ภาพ (โครงเดียวกับแล็บ 2, เทรนบน CPU)

DCGAN ของเรา: ภาพใหม่ที่ไม่ได้ลอกใคร



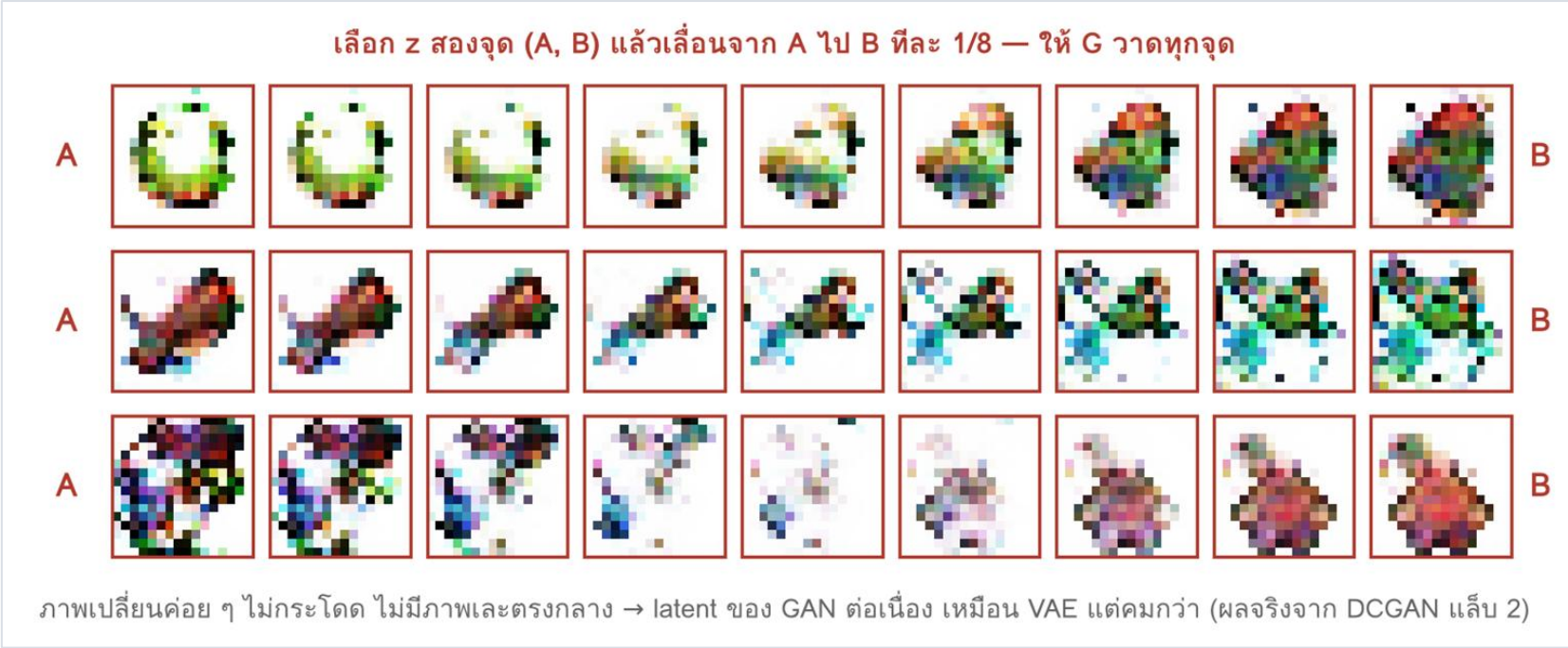
ภาพ: DCGAN ที่เทรนจริงบน sprite 10,000 ภาพ (โครงเดียวกับแล็บ 2, เทรนบน CPU)

latent มี 'ทิศทาง': บวกลบ z แล้วได้ใบหน้าใหม่



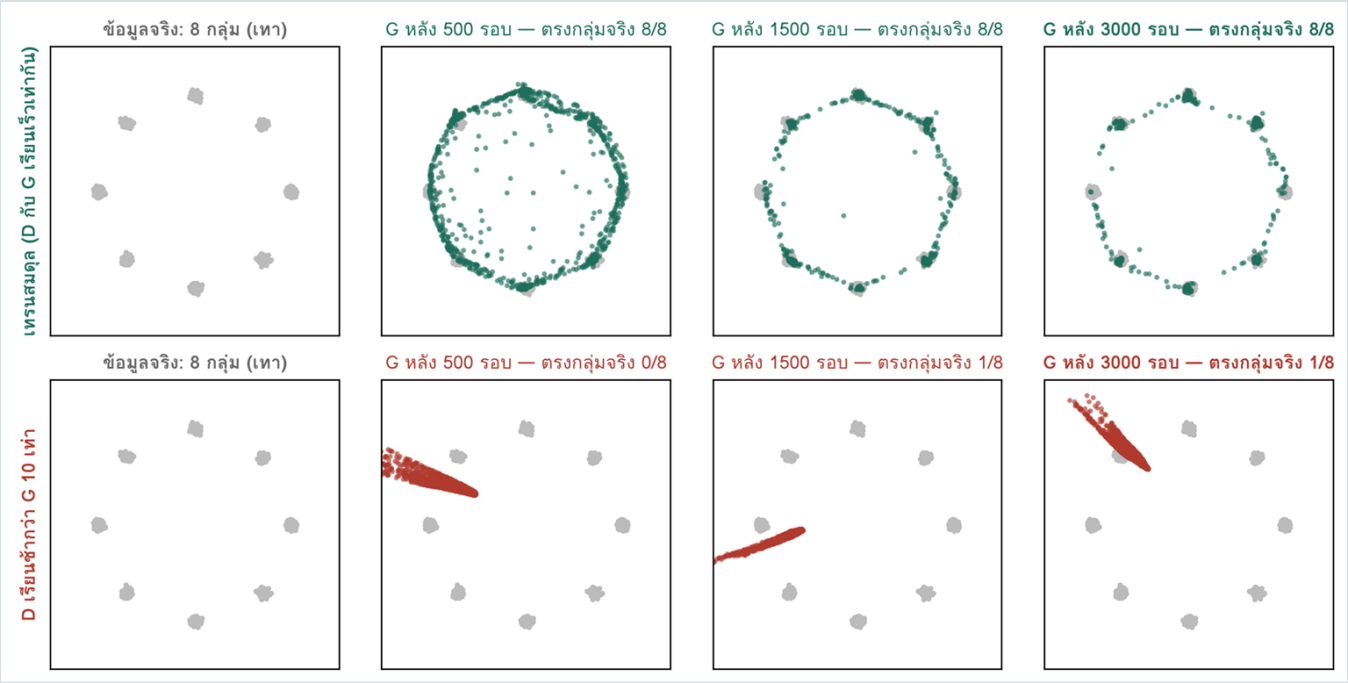
ภาพ: Siyu Zhu (Fudan) Lecture 3 หน้า 38 — ใบหน้าจาก DCGAN (Radford et al. 2016) จัดเรียงใหม่พร้อมป้ายไทย

เดินใน latent ของ GAN: ภาพเปลี่ยนต่อเนื่อง ไม่กระโดด



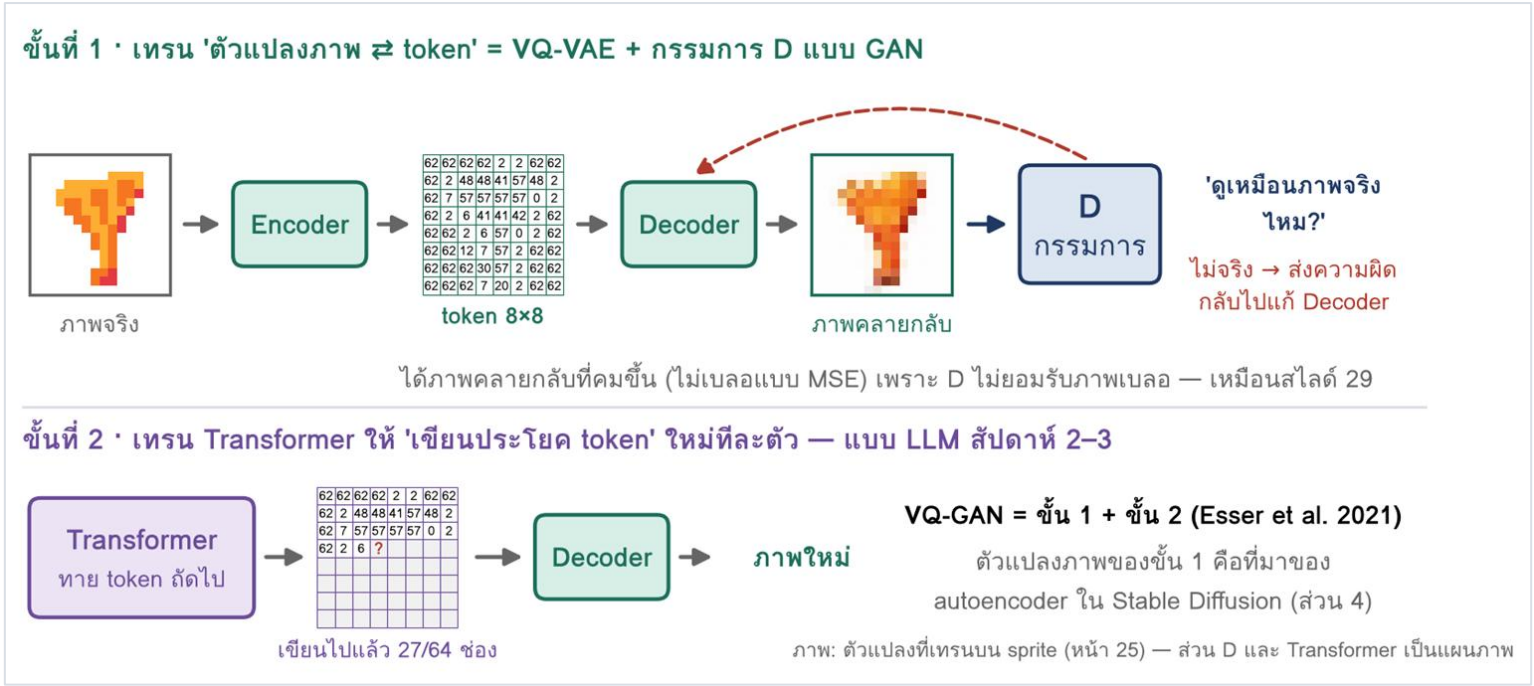
ภาพ: DCGAN ที่เทรนจริงบน sprite 10,000 ภาพ (โครงเดียวกับแล็บ 2, เทรนบน CPU)

ปัญหาของ GAN: mode collapse — ผลิตอยู่แบบเดียว



ภาพ: GAN ขนาดเล็กที่เทรนจริงบนข้อมูล 2 มิติ 8 กลุ่ม (CPU) — ตั้งค่าเหมือนกันทุกอย่าง ต่างแค่ความเร็วการเรียนของ D

VQ-GAN: VQ-VAE + กรรมการ GAN + Transformer



ภาพ: ตัวแปลงภาพ ⇄ token ที่เทรนจริงบน sprite (CPU) + แผนภาพ — แนวคิดจาก Esser et al. 2021 (Siyu Zhu Lecture 3 หน้า 44)

ตระกูล GAN ที่ควรรู้จักชื่อ

ชื่อ	เพิ่มอะไร	ใช้ทำอะไร
GAN (2014)	เกมสองฝ่ายดั้งเดิม	ภาพเล็ก ๆ
DCGAN (2015)	ใช้ CNN + ทริกให้เทรนนิ่ง	ต้นแบบที่เราใช้ในแล็บ 2
Conditional GAN	ป้อนป้าย/เงื่อนไขให้ทั้ง G และ D	สั่งได้ว่าจะเอาคลาสไหน
StyleGAN (2019–)	คุม 'สไตล์' แยกชั้น หน้าคนสมจริง	ใบหน้า, งานศิลป์
BigGAN (2019)	ขยายขนาดใหญ่มาก	ImageNet 512×512
VQ-GAN (2021)	VQ-VAE + กรรมการ GAN + perceptual loss (เทียบฟีเจอร์แทนพิกเซล)	สูตรเทรน autoencoder ของ Stable Diffusion



ภาพที่ CycleGAN แปลง: ม้า ↔ ม้าลาย



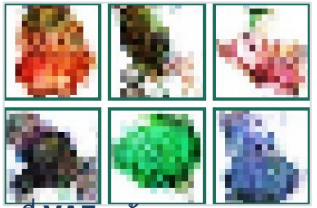
ภาพที่ BigGAN สร้าง (2019)

- GAN ไม่ได้ตายไป: decoder ของ Stable Diffusion เทรนด้วย 'กรรมการ' แบบ GAN (สูตรจาก VQ-GAN)

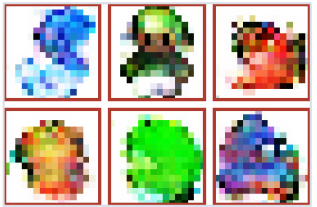
ภาพ: Siyu Zhu (Fudan) Lecture 3 หน้า 40, 41 · ดัดแปลง Stanford University 2024

VAE vs GAN — สรุปก่อนไปต่อ

	VAE	GAN
วิธีสุ่ม	z → decoder ทีเดียว	z → G ทีเดียว
สัญญาณที่ใช้เทรน	เทียบพิกเซล (MSE) + KL	คำตัดสินของ D
ภาพ	เบลอ	คม
ความหลากหลาย	ดี (latent เต็มวงกลม)	เสี่ยง mode collapse
การเทรน	นิ่ง loss ลดเรียบ	แกว่ง ต้องจูนมาก
มี encoder ไหม	มี — แปลงภาพเป็น z ได้	ไม่มี — ย้อนจากภาพไป z ยาก



ภาพที่ VAE สร้าง: หลากหลายแต่เบลอ



ภาพที่ GAN สร้าง: คม

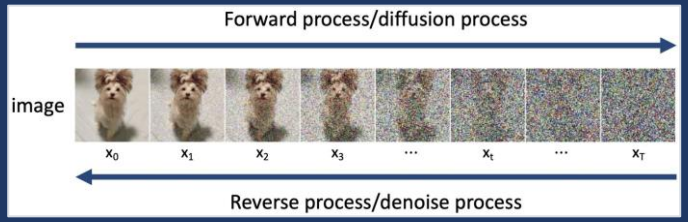
● คำถามนำไปส่วน 4: เอาข้อดีของทั้งสองฝั่งมารวมกันได้ไหม?

ภาพ: VAE และ DCGAN ที่เทรนจริงบน sprite ชุดเดียวกัน (CPU)

ส่วนที่ 4

Diffusion — เดินจาก noise ทีละก้าว

Forward เติม noise · Reverse ทาย noise · เทรนเหมือน regression · DDPM 1,000 ก้าว → DDIM 50 ก้าว · ทำไมชนะ GAN



ภาพ: Siyu Zhu (Fudan) Lecture 4&5 หน้า 12 · ต้นฉบับ Mike Shou, NUS

ไอเดียเดียว: 'ทำลายช้า ๆ แล้วเรียนวิธีกู้คืน'

Forward (เติม noise): สูตรตายตัว ไม่ต้องเรียน

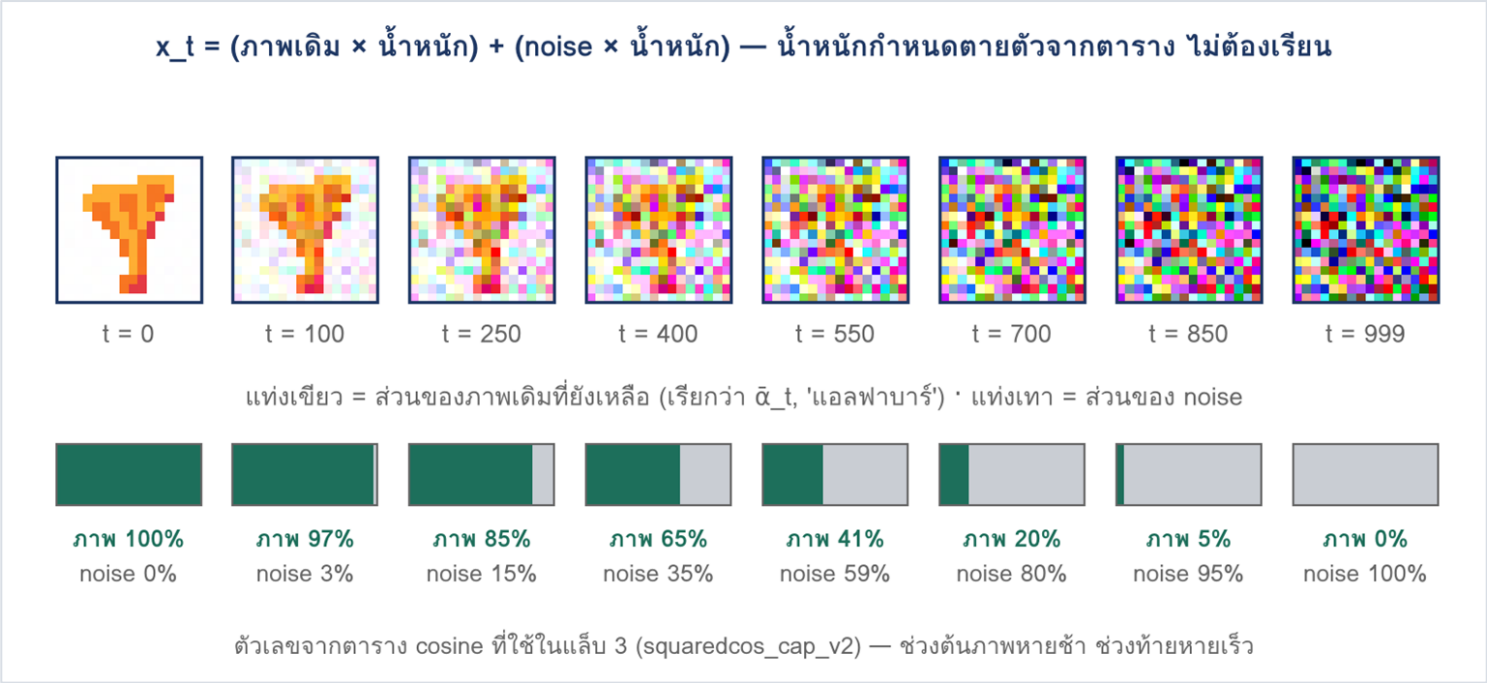


Reverse (ลบ noise): U-Net ที่เทรนแล้ว เริ่มจาก noise สุ่ม ค่อย ๆ ลบจนได้ภาพใหม่



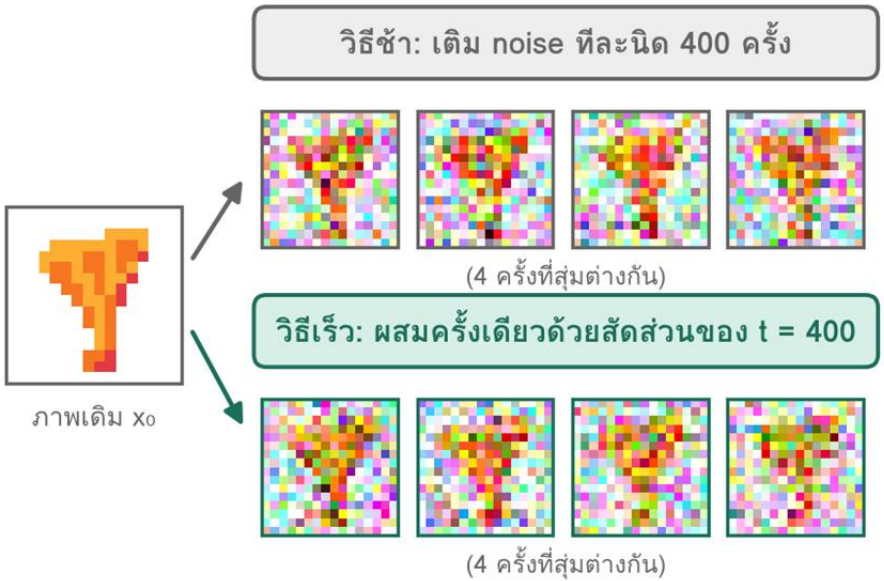
ภาพ: ผลรันจริงของแล็บ 3–4 (diffusion บน sprite 10,000 ภาพ, CPU)

Forward: ทุก t คือภาพเดิมผสม noise ในสัดส่วนที่ตายตัว



ภาพ: สูตร forward จริงของแล็บ 3 บนภาพ sprite

ทริกตอนเทรน: กระโดดไป t ไหนก็ได้ในครั้งเดียว



หน้าตาเหมือนกัน

(ทางสถิติเท่ากันพอดี)

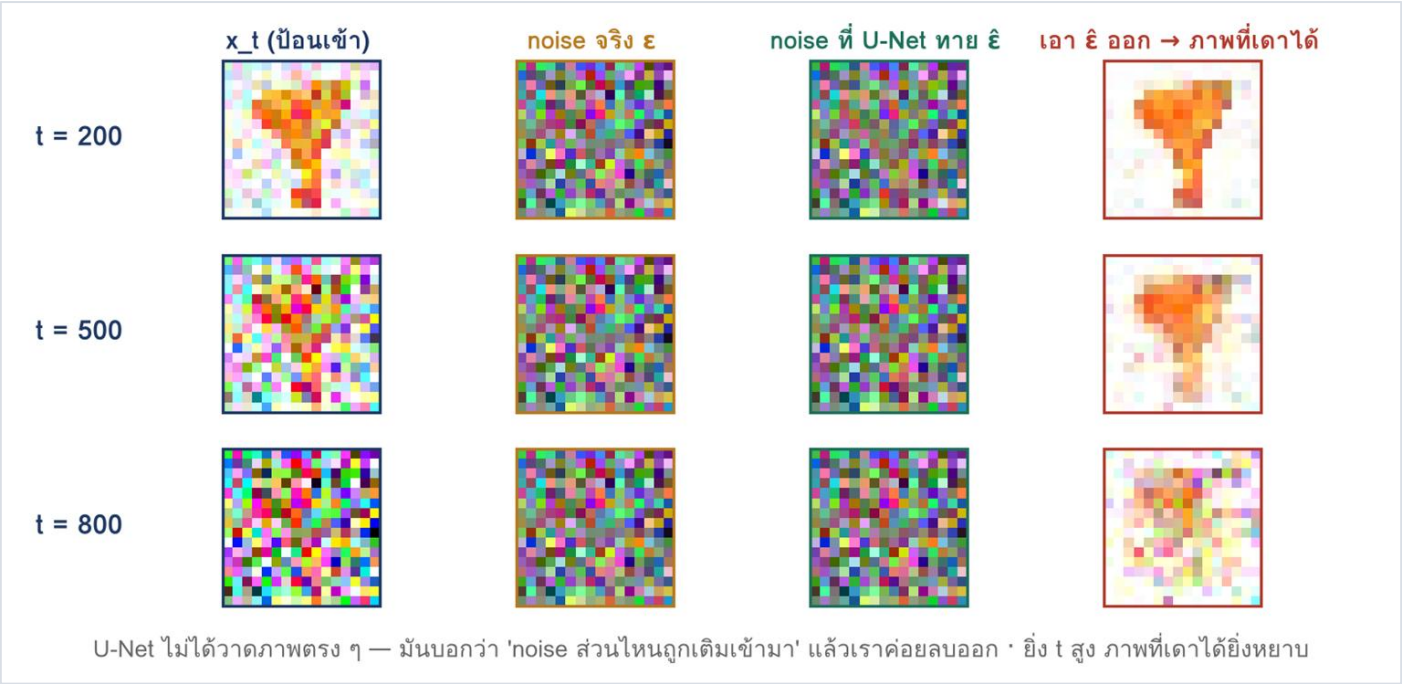
ตอนเทรนจึงกระโดดไป t ไหนก็ได้ในคำสั่งเดียว:

```
noise_sched.add_noise(x0, noise, t)
```

ไม่ต้องวน 400 ครั้งทุกภาพ → เทรนเร็ว

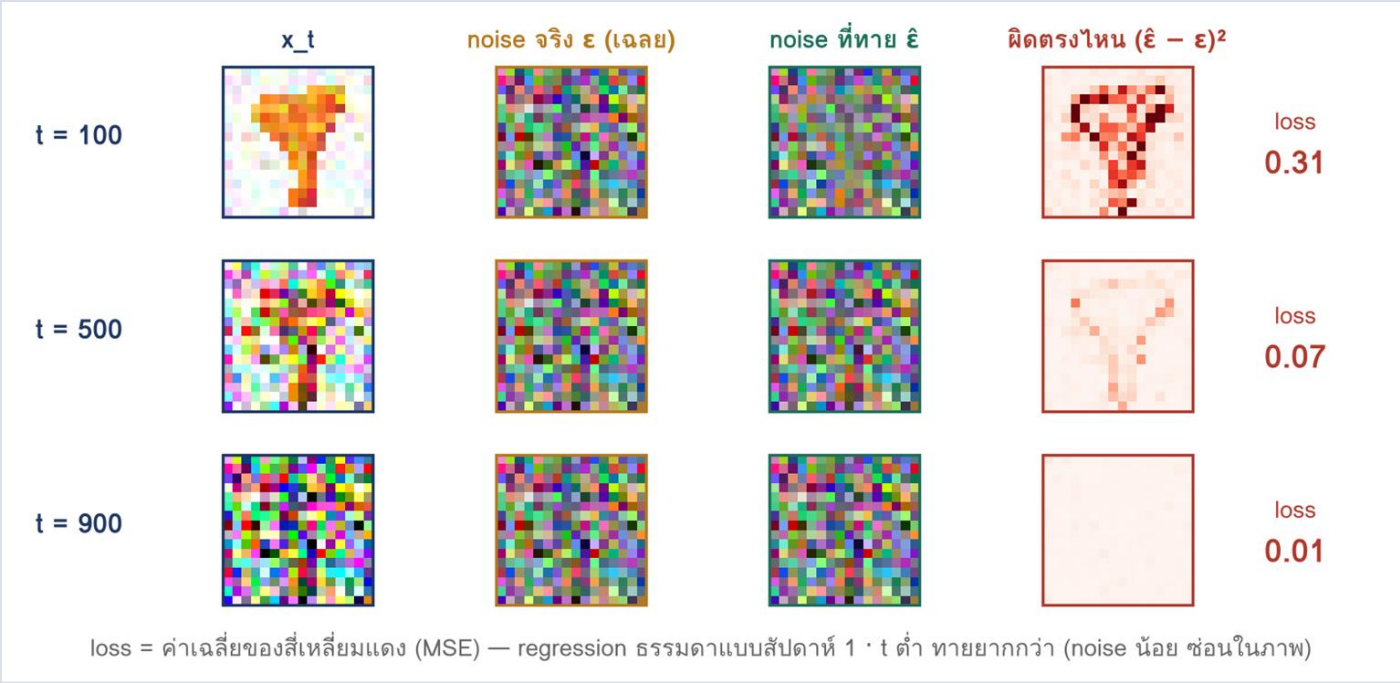
ภาพ: ผลรันจริงของแล็บ 3–4 (diffusion บน sprite 10,000 ภาพ, CPU)

Reverse: เครือข่ายทาย 'noise' ไม่ใช่ทาย 'ภาพ'



ภาพ: U-Net แบบแล็บ 3 ที่เทรนสั้น ๆ (4 epoch) บน sprite บน CPU — ค่าจริงจากโมเดล

loss = ทาย noise ผิดไปเท่าไร (เฉลยคือ noise ที่เราเติมเอง)



ภาพ: U-Net แบบแล็บ 3 ที่เทรนสั้น ๆ (4 epoch) บน sprite บน CPU — ค่าจริงจากโมเดล

เทรน = เกมทาย noise: หนึ่ง batch หน้าตาแบบนี้

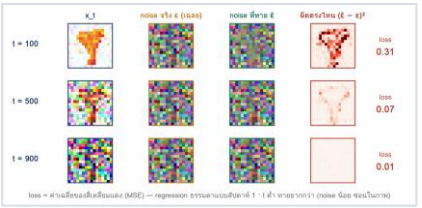


ภาพ: ผลรันจริงของแล็บ 3–4 (diffusion บน sprite 10,000 ภาพ, CPU)

โค้ดที่จะเทรนตอนบ่าย (แล็บ 3)

```
t = torch.randint(0, T, (xb.size(0),), device=device)    # ก้าวสุ่มต่อภาพ
noise = torch.randn_like(xb)
x_noisy = noise_sched.add_noise(xb, noise, t)             # forward 1 บรรทัด
if cond:   # แล็บ 4: สุ่มซ่อนป้าย (ใส่ 5 = "ไม่บอก")
    drop = torch.rand(len(yb), device=device) < p_uncond
    yb = torch.where(drop, torch.full_like(yb, 5), yb)
    pred = model(x_noisy, t, class_labels=yb).sample
else:
    pred = model(x_noisy, t).sample
loss = F.mse_loss(pred, noise)                            # ทาย noise ให้แม่น
opt.zero_grad(); loss.backward(); opt.step(); sched.step()
losses.append(loss.item())
```

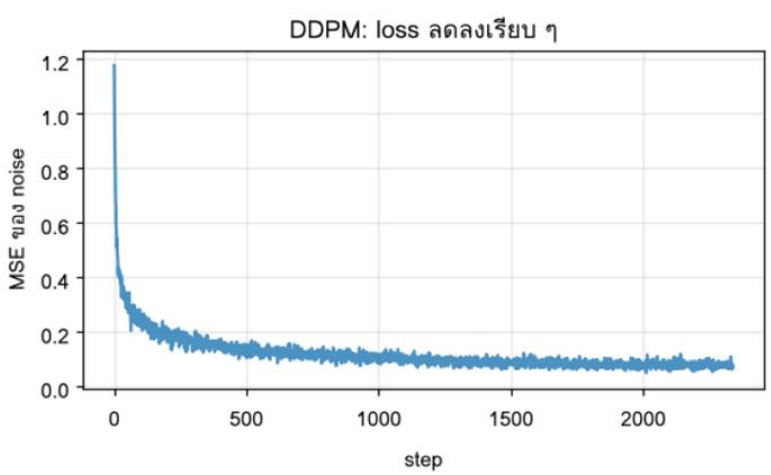
- **t = สุ่มก้าว 0–999 คนละค่าต่อภาพ**
- **noise_sched.add_noise = สูตร forward บรรทัดเดียว**
- model(x_noisy, t).sample = U-Net ทาย noise
- **F.mse_loss(pred, noise) = loss ตัวเดียว**
  - cond=True คือแล็บ 4 (ป้อนป้ายกลุ่ม) — ข้ามไปก่อน
  - เทียบสัปดาห์ 1: นี่คือ training loop ของ regression ปกติ



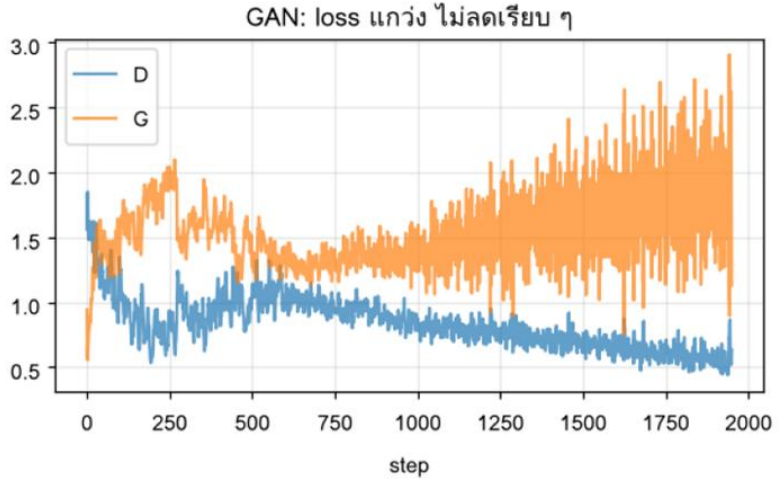
ภาพ: diffusion ที่เทรนจริงบน sprite (หน้าก่อน)

เทรนนิ่งกว่า GAN: loss ลดเรียบและอ่านความหมายได้

Diffusion: loss ลดเรียบ อ่านได้ว่าเก่งขึ้น

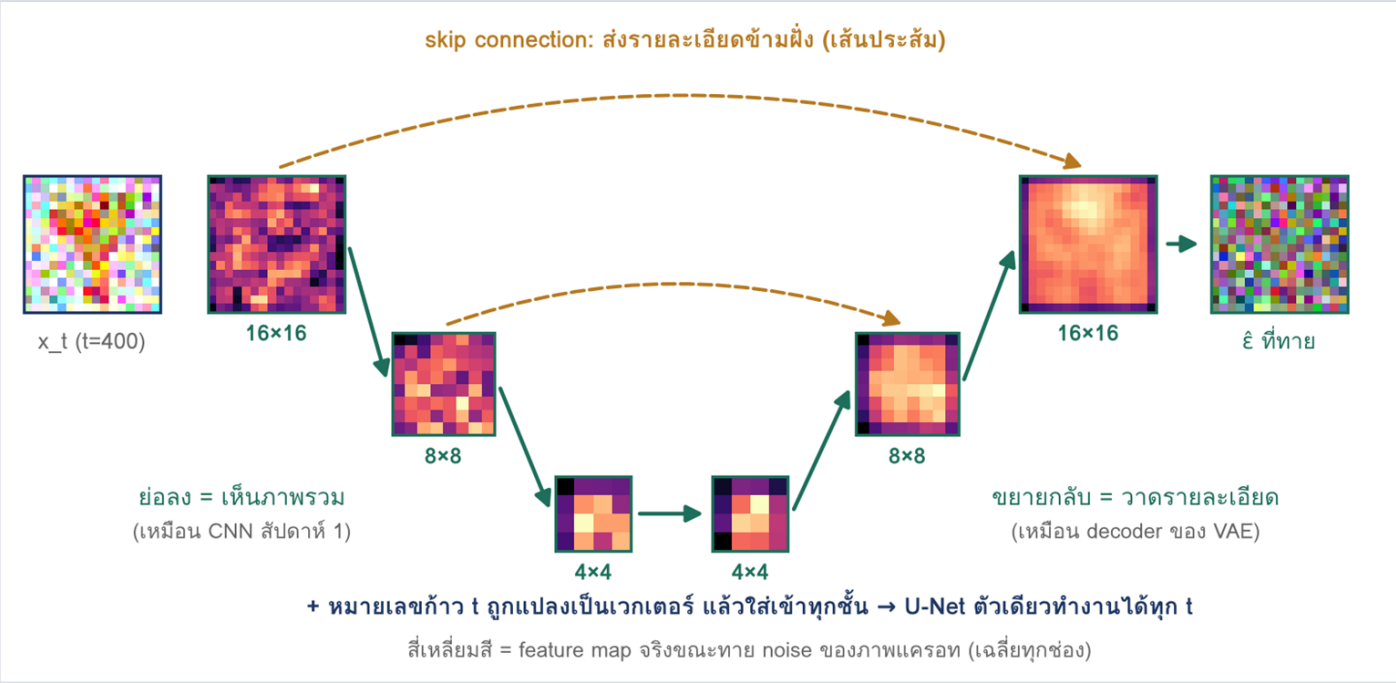


GAN: loss สองฝ่ายแกว่ง บอกแค่ใครได้เปรียบ



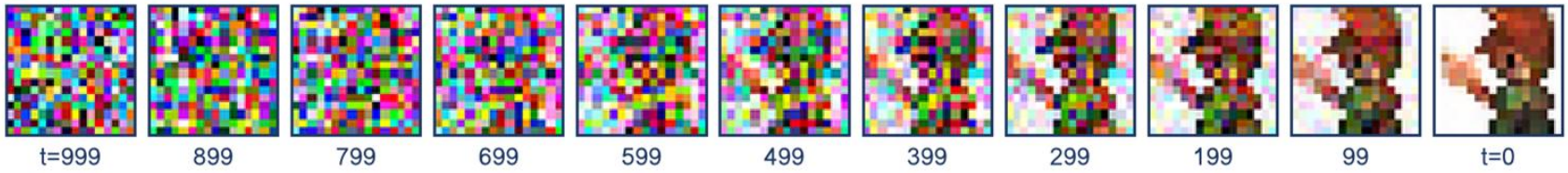
ภาพ: ผลรันจริงของแล็บ 3–4 (diffusion บน sprite 10,000 ภาพ, CPU) · GAN จากสไลด์ 32

U-Net: ตัวทาย noise (ข้างในของจริง)



ภาพ: U-Net แบบแล็บ 3 ที่เทรนสั้น ๆ (4 epoch) บน sprite บน CPU — ค่าจริงจากโมเดล

Sampling: กรอกลับจาก noise ล้วน 1,000 ก้าว (DDPM)

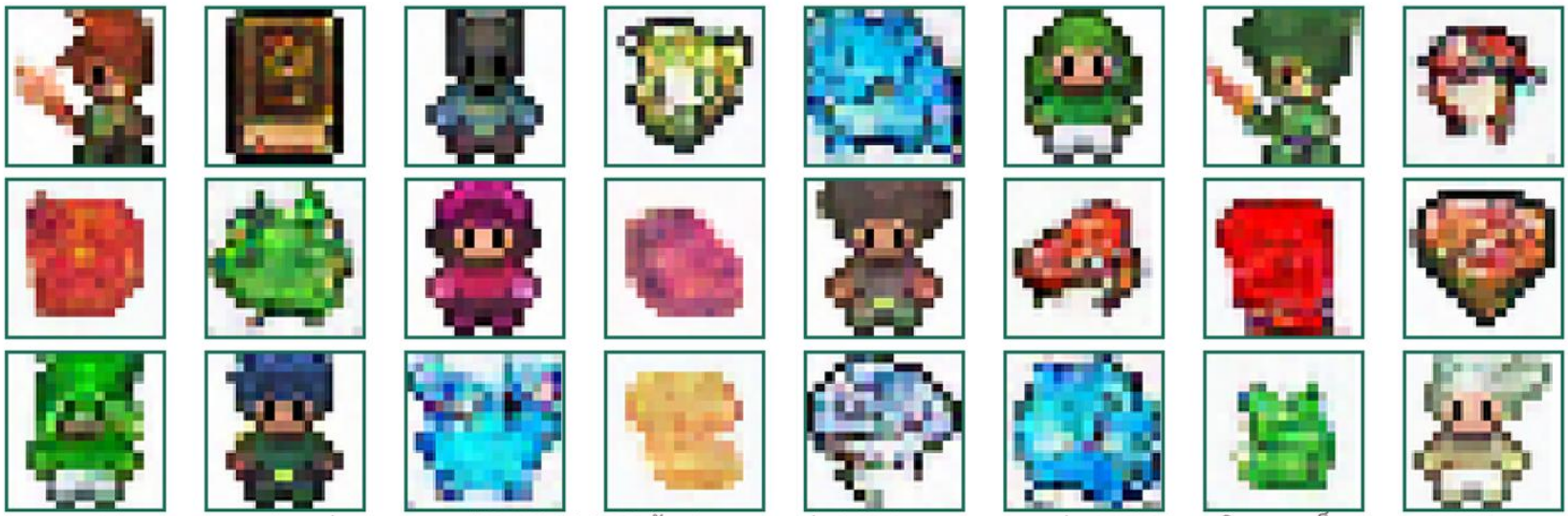


ทุกก้าว: U-Net ทาย noise → ลบออกนิดหนึ่ง → เติม noise ใหม่เล็ก ๆ → วนจาก t = 999 ลงไปถึง 0 (1,000 รอบ)

t สูง = วาดโครงใหญ่ (สี ทรง) · t ต่ำ = เก็บรายละเอียด (ขอบ ตา มือ)

ภาพ: ผลรันจริงของแล็บ 3–4 (diffusion บน sprite 10,000 ภาพ, CPU)

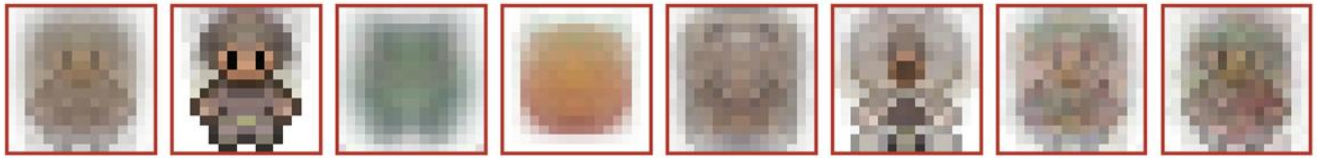
ภาพที่ diffusion ของเราสร้าง (DDPM 1,000 ก้าว)



ภาพ: ผลรันจริงของแล็บ 3–4 (diffusion บน sprite 10,000 ภาพ, CPU)

ทำไมไม่เบลอ และทำไมต้องเติม noise กลับ

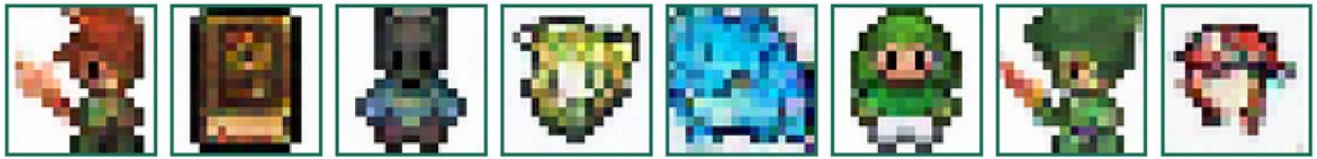
① ถ้าให้ทายภาพสุดท้ายทีเดียวจาก noise ล้วนด้วย MSE → คำตอบที่ผิดน้อยสุดคือ 'ค่าเฉลี่ย' ของภาพที่เป็นไปได้



ก้อนเบลอ ไม่ใช่ภาพ

(ค่าเฉลี่ยของ sprite จริง)

② DDPM: เดิน 1,000 ก้าว — ทายแค่ noise ก้าวเล็ก ๆ + เติม noise กลับเล็กน้อยทุกก้าว



คม และหลากหลาย

(ผลจริงแล็บ 3)

ก้าวเล็ก ๆ = คำตอบที่ถูกเหลือแบบเดียวในแต่ละก้าว จึงไม่ต้องเฉลี่ย · noise ที่เติมกลับทำให้แต่ละรอบเลือกทางต่างกัน → หลากหลาย

ภาพ: ผลรันจริงของแล็บ 3–4 (diffusion บน sprite 10,000 ภาพ, CPU)

DDIM: โมเดลเดิม เดินก้าวใหญ่ขึ้น เร็วขึ้นหลายสิบเท่า



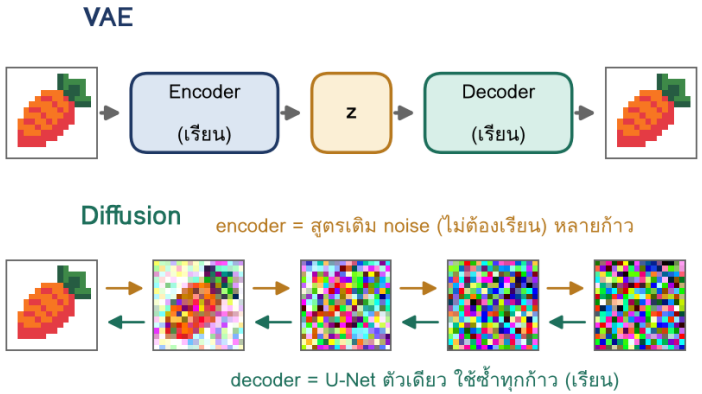
ภาพ: ผลรันจริงของแล็บ 3–4 (diffusion บน sprite 10,000 ภาพ, CPU)

noise เดิม → DDIM ได้ภาพเดิม ไม่ว่าจะเดินกี่ก้าว



ภาพ: ผลรันจริงของแล็บ 3 (diffusion บน sprite 10,000 ภาพ, CPU)

Diffusion = VAE ที่มีหลายชั้นซ้อนกัน



Diffusion =

VAE หลายชั้นซ้อนกัน

- encoder ตายตัว
- latent ขนาดเท่าภาพ
- decoder ตัวเดียว ใช้ซ้ำทุกชั้น

- VAE: encoder ที่ต้องเรียน บีบเป็น z เล็ก ๆ → decoder คลายทีเดียว
- Diffusion: encoder = สูตรเดิม noise ตายตัว (ไม่ต้องเรียน) หลายก้าว, latent ขนาดเท่าภาพ, decoder = U-Net ตัวเดียวใช้ซ้ำทุกก้าว
- เพราะ encoder ตายตัว จึงไม่มี 'รู' และไม่ต้องประนีประนอมระหว่าง MSE กับ KL

ทำไม Diffusion ชนะ GAN (2021–ปัจจุบัน)

1 เทรนนิ่ง

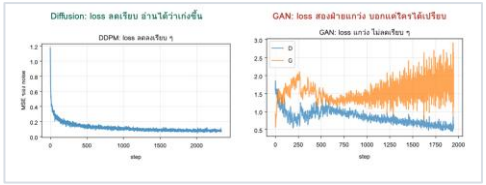
loss ตัวเดียว (MSE ของ noise) ลดลงเรียบ ๆ
ไม่มีสองฝ่ายไล่กัน ไม่ต้องจูนทริกพิเศษ
ขยายขนาดโมเดล/ข้อมูลได้ตรง ๆ (scale ได้)

2 หลากหลาย

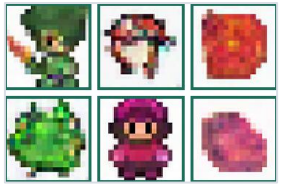
ไม่มี mode collapse
ทุกภาพจริงถูก 'ทำลาย' และ 'กู้คืน' ตอนเทรน → โมเดลต้องครอบคลุมทุกแบบ
ความหลากหลายมาจาก noise ตั้งต้นโดยตรง

3 ควบคุมง่าย

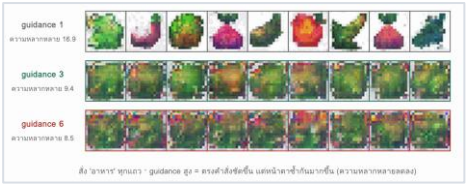
ใส่เงื่อนไข (ป้าย ข้อความ ภาพร่าง) เข้า U-Net ได้ตรง ๆ
แก้ไขภาพได้โดยเริ่มจากกลางทาง (img2img)
เปลี่ยนวิธีเดิน (DDIM) ได้โดยไม่เทรนใหม่



loss ลดเรียบ (เทรนจริง)



ภาพจาก diffusion ของเรา



ปรับ guidance ได้

ภาพ: diffusion ที่เทรนจริงบน sprite (CPU)

ราคาที่ต้องจ่าย: ช้า — และ 3 วิธีแก้

1 DDIM / sampler ใหม่

ลดจาก 1,000 ก้าวเหลือ 20–50 โดยไม่เทรนใหม่
(ทำวันนี้ในแล็บ 3)

sampler รุ่นใหม่ (DPM-Solver, Euler) ลงได้ถึง ~10 ก้าว

2 Latent diffusion

เอา VAE บีบภาพก่อน 512×512 → 64×64
แล้วค่อย diffuse บน latent

เร็วขึ้นหลายสิบเท่า = Stable Diffusion
(สัปดาห์หน้า)

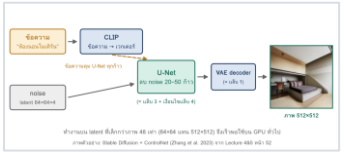
3 Distillation

เทรนโมเดลลูกให้เดิน 1–4 ก้าว เลียนแบบโมเดลแม่ 50 ก้าว

SD-Turbo, SDXL-Lightning, Flux Schnell
(กล่าวถึงสัปดาห์หน้า)



DDPM vs DDIM (เวลาจริงบน CPU)



Latent diffusion: ทำบน latent

ภาพ: diffusion ที่เทรนจริงบน sprite (CPU) · ภาพห้องนอนจาก Lecture 4&5 หน้า 52

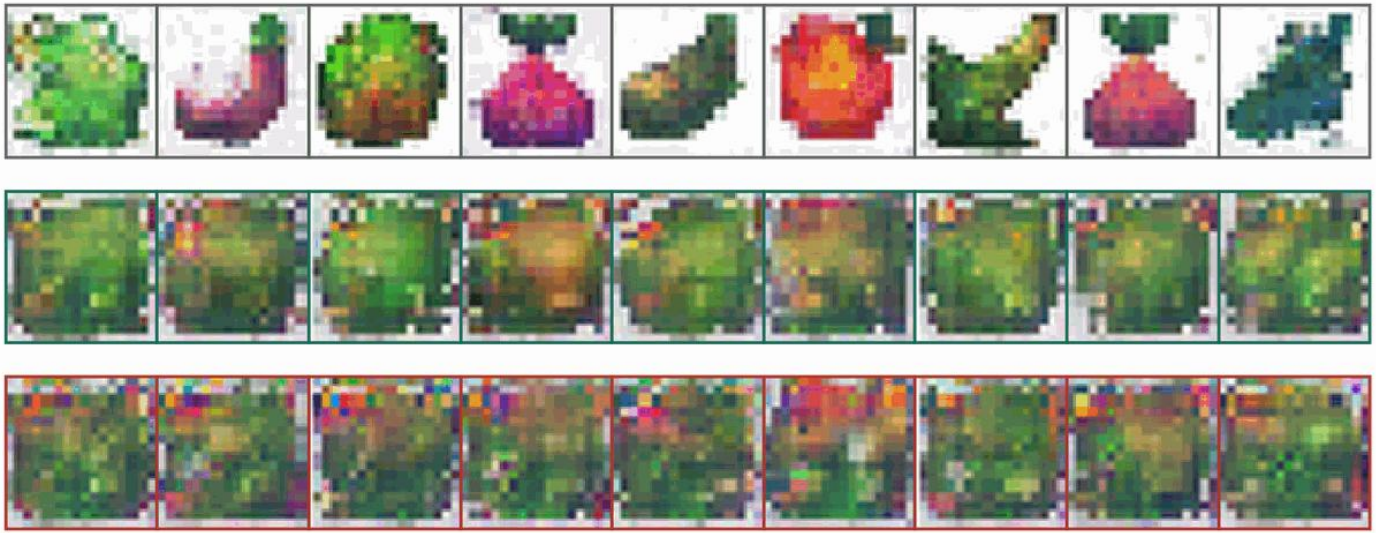
ใส่เงื่อนไข: วันนี้ป้ายกลุ่ม สัปดาห์หน้าข้อความ



U-Net ตัวเดียว เปลี่ยนแค่ป้ายกลุ่มที่ป้อน → ได้ของคนละกลุ่ม (ผลจริงจากแล็บ 4)

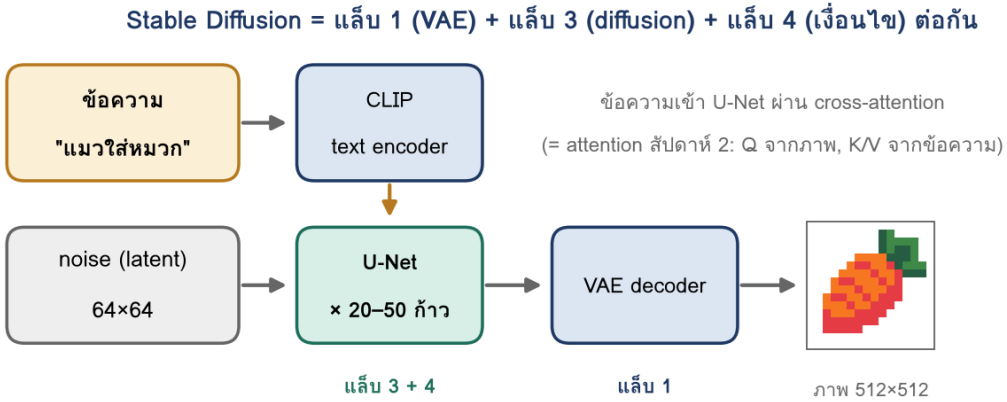
ภาพ: ผลรันจริงของแล็บ 3–4 (diffusion บน sprite 10,000 ภาพ, CPU)

ปุ่ม guidance: ตรงคำสั่งมากขึ้น แต่หลากหลายน้อยลง



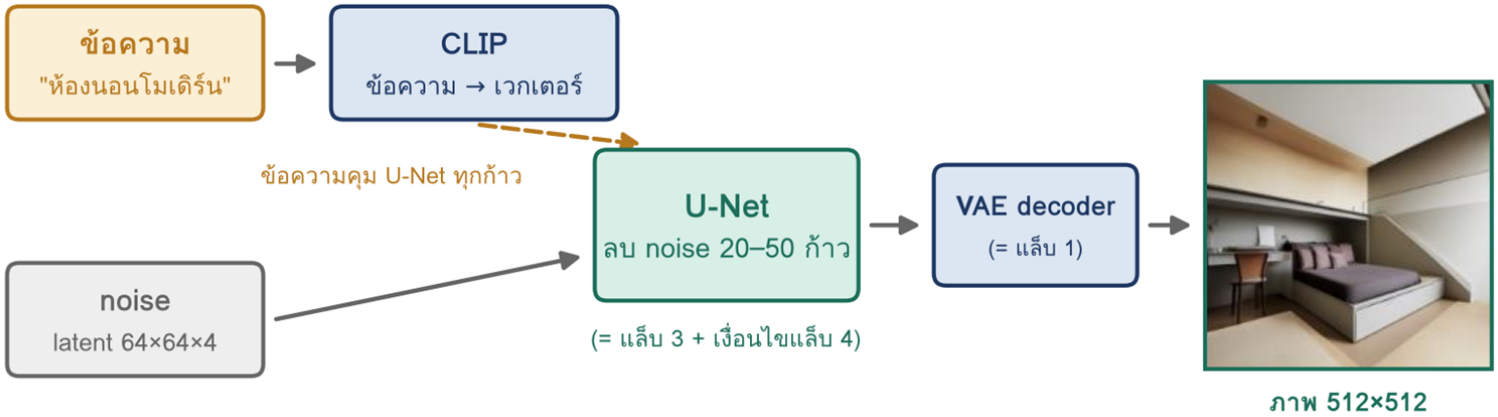
ภาพ: ผลรันจริงของแล็บ 3–4 (diffusion บน sprite 10,000 ภาพ, CPU)

ประกอบร่าง: Stable Diffusion = 3 แล็บของวันนี้ต่อกัน



- แล็บ 1 (VAE): บีบภาพ 512×512 เป็น latent 64×64 และคลายกลับ
- แล็บ 3 (diffusion): U-Net ทาย noise บน latent 20–50 ก้าว (DDIM)
- แล็บ 4 (เงื่อนไข): แทนป้ายกลุ่มด้วยข้อความผ่าน CLIP + cross-attention (= attention สัปดาห์ 2: Q จากภาพ K/V จากข้อความ)

Stable Diffusion: 3 ชิ้นของวันนี้ ทำงานบน latent



ทำงานบน latent ที่เล็กกว่าภาพ 48 เท่า (64×64 แทน 512×512) จึงเร็วพอใช้บน GPU ทั่วไป

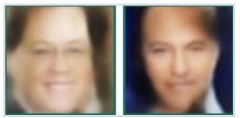
ภาพตัวอย่าง: Stable Diffusion + ControlNet (Zhang et al. 2023) จาก Lecture 4&5 หน้า 52

ภาพ: แผนภาพย่อจาก Rombach et al. 2022 · ภาพห้องนอนจาก ControlNet (Siyu Zhu Lecture 4&5 หน้า 46, 52)

สรุป

สรุป 3 โมเดล — กลับมาดูตารางตอนเช้าอีกครั้ง

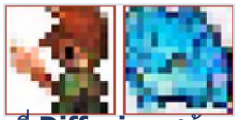
	VAE	GAN	Diffusion
สุ่มภาพอย่างไร	z → decoder ทีเดียว	z → G ทีเดียว	noise → ลบทีละก้าว
loss	MSE + KL (นิ่ง)	สองฝ่ายสู้กัน (แกว่ง)	MSE ของ noise (นิ่ง)
ภาพ	เบลอ	คม	คม
ความหลากหลาย	ดี	เสี่ยง mode collapse	ดี
สร้างเร็ว	เร็วมาก	เร็วมาก	ช้า → DDIM / latent
อยู่ตรงไหนใน SD	ตัวบีบภาพ	กรรมการของ decoder	หัวใจ (U-Net)



ภาพที่ VAE สร้าง: เบลอ



ภาพที่ GAN สร้าง: คม



ภาพที่ Diffusion สร้าง: คม หลากหลาย

● ตอนบ่ายจะเทรนทั้งสามบนชุดข้อมูลเดียวกัน แล้วดูว่าตารางนี้จริงไหม

ภาพ: Siyu Zhu (Fudan) Lecture 2 หน้า 81 · Lecture 3 หน้า 33 · Lecture 4&5 หน้า 52

ช่วงบ่าย: 4 แล็บบน Colab T4 — ชุดข้อมูล Sprites 16×16

1 แล็บ 1 · VAE

เทรน ~1 นาที
ดูภาพคลายกลับ (เบลอ)
สุ่มภาพใหม่ · interpolate
แผนที่ latent แยกกลุ่มเองได้

2 แล็บ 2 · GAN

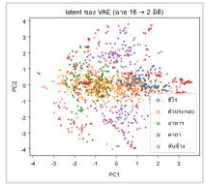
เทรน ~2 นาที
ดู loss แกว่ง
ภาพคมขึ้นทุก 5 epoch
วัดความหลากหลาย vs ข้อมูลจริง

3 แล็บ 3 · DDPM/DDIM

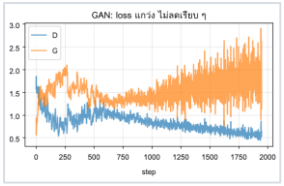
ดู forward noising
เทรน ~4 นาที loss นิ่ง
ดูเส้นทาง denoising
1,000 vs 50 vs 20 vs 10 ก้าว

4 แล็บ 4 · เงื่อนไข

ป้อนป้ายกลุ่มเข้า U-Net
สั่ง 'อาหาร' 'ฮีโร่' ได้
หมุน guidance 1/3/6
= ปุ่ม CFG ของ SD



แล็บ 1: แผนที่ latent



แล็บ 2: loss แกว่ง



แล็บ 3: sprite จาก DDPM



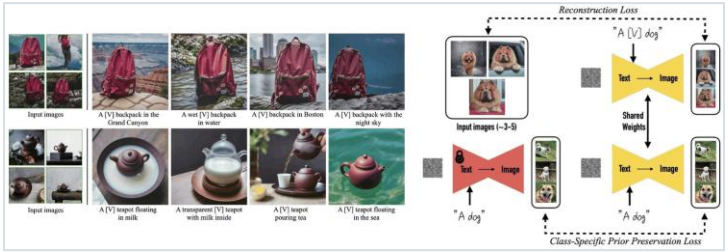
แล็บ 4: สั่งทีละกลุ่ม

ภาพผลลัพธ์จากการรัน notebook แล็บ (CPU)

ปิดช่วงเช้า

คำถามชวนคิดก่อนพัก

ถ้าอยากได้โมเดลที่ 'สร้างภาพสินค้าของบริษัทตัวเอง' จากข้อความ ควรเทรนตั้งแต่ศูนย์แบบวันนี้ หรือเอา Stable Diffusion มาปรับ (เหมือน LoRA สัปดาห์ 3)?



คำใบ้: สัปดาห์ 3 เราสรุปว่า 'prompt + RAG ก่อน, fine-tune เมื่อจำเป็น' — สัปดาห์หน้าจะได้คำตอบฝั่งภาพ: prompt / ControlNet / LoRA

ภาพ: Siyu Zhu (Fudan) Lecture 4&5 หน้า 49 · ต้นฉบับ Mike Shou, NUS